UTN Facultad Regional La Rioja

Trabajo Final
“Proyecto”

Índice

Requisitos comunes, opciones de proyecto y criterios de entrega.

1 Presentación y propósito integrador ... 3
2 Organización de los equipos ... 4
3 Arquitectura mínima esperada ... 5
4 Desarrollo de las quince opciones ... 7
4.1 Kiosco o comercio minorista ... 8
4.2 Cine ... 13
4.3 Biblioteca ... 18
4.4 Agencia de transporte ... 23
4.5 Complejo deportivo ... 28
4.6 Restaurante o bar ... 33
4.7 Gimnasio ... 38
4.8 Concesionaria ... 43
4.9 Taller mecánico ... 48
4.10 Hotel ... 53
4.11 Torneo de tatetí ... 58
4.12 Gestión clínica ... 63
4.13 Batalla naval ... 68
4.14 Ahorcado ... 73
4.15 Trivia tipo Preguntados ... 78
5 Roles y responsabilidad compartida ... 83
6 Requisitos funcionales obligatorios ... 84
7 Evidencias específicas de Programación II ... 85
8 Evidencias específicas de Base de Datos I ... 87
9 Entregables ... 89
10 Criterios mínimos para GitLab ... 90
11 Rúbrica porcentual propuesta ... 91
12 Cronograma de avances ... 95
13 Organización de la exposición de diez minutos ... 96
14 Lista final de verificación ... 97

1 Presentación y propósito integrador

El equipo deberá desarrollar una aplicación de escritorio en **C++ con Qt Widgets**, conectada con una **base de datos relacional**, que resuelva una de las quince situaciones propuestas. El trabajo integra diseño orientado a objetos, interfaz gráfica, persistencia de datos y documentación profesional. Se espera un sistema funcional, comprensible y demostrable, cuyo alcance pueda completarse entre cuatro estudiantes dentro del cronograma.

Una solución de extremo a extremo permite iniciar una acción en la interfaz, aplicar las reglas del negocio, guardar sus efectos y volver a consultar el resultado. Por ejemplo, confirmar una venta debe validar los productos y las cantidades, registrar el comprobante y sus detalles, actualizar las existencias y mostrar el resultado. Una pantalla que informa éxito sin guardar los datos, o una base cargada manualmente sin integración con la aplicación, no completa ese recorrido.

1.1 Objetivos por espacio curricular

Espacio	Aprendizajes que deberán evidenciar
Programación II	Aplicar clases, encapsulamiento, composición y asociaciones; herencia y polimorfismo cuando correspondan; colecciones y algoritmos de la STL; manejo de errores y excepciones; separación clara entre interfaz, lógica y datos.
Base de Datos I	Modelar información del negocio, definir entidades y relaciones, asegurar integridad, implementar CRUD y documentar la estructura y las consultas. CRUD comprende alta, consulta, modificación y baja de registros.

1.2 Cómo interpretar la consigna

Obligatorio. Comprende los requisitos comunes de las secciones 2 a 14 y el alcance funcional de la opción asignada. Las reglas académicas indicadas en cada ficha delimitan una implementación sencilla y comprobable. Deben quedar reflejadas en los requisitos, el código y las pruebas del equipo.

Orientativo. Los nombres de clases, entidades, carpetas, widgets, contenedores y consultas muestran posibilidades de diseño. No constituyen un modelo resuelto ni una cantidad mínima exigida. El equipo deberá seleccionar y justificar su propia solución.

Opcional. Las ampliaciones se identifican expresamente. No son nuevas condiciones de aprobación ni reemplazan funcionalidades obligatorias. Primero se deberá completar e integrar el alcance central.

Las operaciones de cobro, emisión de comprobantes y financiación se implementarán como registros internos con datos de prueba. No se exigen facturación fiscal, pasarelas de pago, conexiones bancarias ni otros servicios externos.

1.3 Material de referencia de Programación II

El trabajo recupera las unidades de introducción a C++, conceptos básicos de POO, encapsulamiento, herencia y polimorfismo; manejo de excepciones; STL y contenedores. Las clases prácticas de Qt aportan introducción a Widgets, señales y slots, tipos de ventanas, layouts, estilos, QMainWindow y compilación. El material de archivos puede orientar una importación o exportación complementaria; no reemplaza la base relacional requerida.

2 Organización de los equipos

Los equipos estarán integrados por **cuatro estudiantes de la misma comisión**. La conformación se registrará mediante el enlace habilitado en la plataforma. A cada equipo se le asignará una de las quince opciones de proyecto de la sección 4.

Al iniciar, deberán acordar qué problema resuelven, quién utilizará el sistema, cuáles son sus límites y cómo se reconocerá una operación terminada correctamente. Las responsabilidades iniciales deberán quedar registradas junto con la planificación.

2.1 Acuerdos de trabajo obligatorios

- Asignar una responsabilidad principal a cada integrante y establecer colaboración y revisión cruzada. Los roles orientan la organización, pero no delimitan qué partes puede desarrollar o revisar cada persona.
- Utilizar GitLab desde el inicio, con tareas, commits identificables y documentación actualizada. Un commit registra un conjunto de cambios en el historial del proyecto.
- Trabajar de manera coordinada en modelo, lógica, base e interfaz. La integración deberá comenzar con recorridos pequeños y ampliarse en cada avance.
- Presentar avances parciales en las fechas de la sección 12, con evidencia ejecutable acorde con la etapa y documentación consistente.
- Comprender el sistema completo y poder explicar sus decisiones técnicas, más allá de la responsabilidad principal.

La evaluación combinará el resultado grupal con evidencias individuales de participación y dominio. La asignación de un rol, por sí sola, no demuestra un aporte: deberán identificarse las tareas realizadas, sus resultados y su integración en el producto.

2.2 Secuencia inicial orientativa

1. Leer los requisitos comunes y la ficha del tema asignado. Identificar las entidades administrables y el proceso central que se mostrará en la defensa.
2. Definir casos de uso con actor, precondiciones, pasos, alternativas y resultado esperado. Un caso de uso describe una interacción que permite alcanzar un objetivo concreto.
3. Preparar el repositorio, el proyecto base compilable y una primera planificación. Distribuir tareas que produzcan resultados verificables.
4. Elaborar conjuntamente un primer modelo de clases, un modelo de datos y bocetos de pantallas. Revisar que representen el mismo negocio.
5. Implementar un recorrido pequeño que atraviese las capas y luego incorporar las demás operaciones, validaciones y reportes.

Esta secuencia es una guía de organización, no agrega fechas ni entregas al cronograma. Las decisiones y los cambios de alcance deberán mantenerse visibles en la documentación.

3 Arquitectura mínima esperada

La aplicación deberá separar responsabilidades para que una regla del negocio no dependa de un botón específico y una consulta SQL no quede dispersa entre pantallas. Las capas pueden organizarse dentro del mismo proyecto; no se requieren servicios de red ni aplicaciones independientes.

Presentación → Servicios y lógica de negocio → Acceso a datos → Base de datos

Capa	Responsabilidad y límite
Presentación	Qt Widgets: capturar datos, mostrar resultados y mensajes, organizar la navegación, aplicar layouts y hojas de estilo QSS. Invoca servicios y no decide por sí sola reglas como disponibilidad, total o resultado de una partida.
Servicios y lógica de negocio	Clases C++ que representan el dominio y coordinan casos de uso. Aplican reglas y validaciones, utilizan STL y tratan errores y excepciones. Devuelven resultados comprensibles para la interfaz.
Acceso a datos	Encapsular persistencia, consultas y transformación de registros en datos utilizables por la lógica. Informar errores y participar en operaciones consistentes; no mostrar cuadros de diálogo desde esta capa.
Base de datos	Conservar información relacional y proteger su integridad mediante claves, relaciones y restricciones. Se prefiere SQLite o MySQL. CSV se admite solamente como importación o exportación complementaria.
Documentación transversal	Mantener requisitos, casos de uso, diagramas, modelo de datos, pruebas, manual y planificación coherentes con la versión implementada.

3.1 Recorrido orientativo de una venta

1. La persona selecciona productos y cantidades en Qt y solicita confirmar. La pantalla transforma entradas válidas en una solicitud para el servicio.
2. El servicio comprueba productos activos, cantidades positivas y stock disponible; calcula el total con los precios correspondientes a la operación.
3. El acceso a datos registra cabecera, detalles y movimientos de stock dentro de una transacción. Una transacción reúne cambios que deben confirmarse todos juntos o revertirse si alguno falla.
4. La base confirma los cambios. Recién entonces la interfaz muestra el comprobante interno y las existencias actualizadas. Ante un error, informa el problema sin simular una venta exitosa.
5. Al cerrar y volver a abrir la aplicación, la venta puede consultarse y el stock conserva su valor. Si un intento fue rechazado, no debe aparecer una venta parcial.

El mismo principio se aplica a una reserva, un préstamo, una orden de trabajo o el cierre de una partida. Los controles visuales ayudan a prevenir errores; las reglas también deben comprobarse en la lógica y, cuando corresponda, en la base.

3.2 Organización orientativa del proyecto

Ubicación sugerida	Contenido
src/main.cpp	Inicio de la aplicación y preparación de sus componentes.
src/ui/	Ventanas, diálogos, archivos .ui y conexiones de señales y slots.
src/domain/ y src/services/	Clases del dominio y coordinación de casos de uso, con archivos .h y .cpp.
src/data/	Acceso a la base, consultas y operaciones de persistencia.
resources/	QSS, iconos y otros recursos utilizados por la interfaz.
database/	Scripts de creación, datos de prueba y consultas documentadas.
docs/ y tests/	Documentación y evidencias de prueba; código de pruebas cuando se utilice.
README.md y archivos del proyecto	Preparación, compilación, ejecución y configuración necesaria.

Se evaluará la **separación efectiva de responsabilidades**, no la copia literal de esta estructura. Las decisiones deberán poder justificarse mediante un caso de uso real y su recorrido por el código.

3.3 Interfaz y temas visuales

La interfaz deberá ofrecer **dos temas visuales: uno oscuro y otro de elección libre**, aplicados mediante QSS. El cambio deberá mantener legibles textos, controles, selecciones y mensajes en las distintas pantallas. Los layouts deberán permitir redimensionar las ventanas sin superponer ni ocultar controles esenciales. No se requiere incorporar una tecnología de interfaz distinta de Qt Widgets.

4 Desarrollo de las quince opciones

Cada equipo desarrollará solamente la opción asignada, además de todos los requisitos comunes. Las fichas distinguen el alcance obligatorio, las orientaciones de diseño y las ampliaciones opcionales. Los ejemplos de clases y entidades no reemplazan el análisis ni el modelado que debe elaborar el equipo.

4.0 Criterio común de alcance y demostración

En todas las opciones deberá poder observarse un proceso principal completo, administración de las entidades pertinentes mediante CRUD, validación de un caso inválido, conservación de los datos entre ejecuciones y consultas desde la aplicación. Los registros históricos deberán conservar su coherencia al modificar o dar de baja datos maestros.

Las reglas académicas de cada ficha delimitan un sistema pequeño: una sede o establecimiento, operación local y datos ficticios. Cuando la ficha indique que el equipo debe fijar un parámetro, este deberá documentarse y aplicarse de forma consistente. No se exige reproducir todos los procesos de una organización real.

Las consultas sugeridas son orientativas. El equipo deberá elegir reportes pertinentes que permitan revisar la actividad y los resultados del sistema; no se impone un número arbitrario de consultas ni de tablas.

Condiciones comunes de los cuatro juegos

Tatetí, batalla naval, ahorcado y trivia deberán implementar la **lógica jugable**, además de login, Qt Widgets, CRUD de sus entidades administrables, consultas, documentación y los demás requisitos comunes. Un formulario que carga resultados manualmente no reemplaza la implementación del juego.

La modalidad será local, en una misma computadora. Los participantes de una competencia pueden registrarse como jugadores y vincularse con usuarios cuando el diseño lo requiera; no es necesario exigir un inicio de sesión por cada cambio de turno. El acceso a la aplicación sí deberá cumplir el requisito de autenticación.

Se deberán conservar participantes, configuración aplicable, desarrollo necesario para explicar el resultado y estado de la partida. La ficha establece qué registrar y cuándo. Los resultados finalizados alimentarán el historial, las posiciones o el ranking sin duplicarse al volver a abrir la pantalla. Una partida interrumpida se distinguirá de una finalizada y no sumará resultados como si hubiese terminado. La reanudación puede incorporarse como mejora opcional, salvo que el equipo la elija como parte de su solución.

El CRUD corresponde a entidades administrables, como jugadores, categorías, preguntas o competencias aún no iniciadas. No implica editar libremente jugadas ni resultados finalizados. Las correcciones o anulaciones deberán conservar trazabilidad y respetar las reglas del dominio.

La exigencia comparable se construye con juego real, estados y validaciones, administración de información, persistencia y consultas. Red, inteligencia artificial avanzada y servicios externos serán ampliaciones opcionales.

Las ocho secciones de cada ficha organizan el trabajo de los cuatro integrantes; no crean nuevos roles.
Backend/C++, Frontend/Qt, Base de datos y Análisis y gestión mantienen las responsabilidades de la sección 5.
GitLab, documentación, pruebas e integración son tareas compartidas.

4.1 Kiosco o comercio minorista

1. Nombre del tema

Kiosco o comercio minorista. Sistema de gestión de compras, ventas y existencias para un kiosco o comercio minorista, con comprobantes internos y cierre diario.

2. Objetivo

Organizar las compras y ventas de un comercio pequeño. El administrador mantiene los datos y una persona operadora registra movimientos, consulta existencias y realiza el cierre diario.

Alcance obligatorio y límites. Administrar productos, categorías, proveedores y medios de pago; registrar compras y ventas con sus detalles; actualizar automáticamente el stock; mostrar alertas de reposición y generar el cierre diario. El alcance comprende un local, una caja y comprobantes internos. Una compra incorpora unidades y una venta las descuenta; no se exige contabilidad ni facturación fiscal.

Recorrido principal obligatorio. Dar de alta un producto y proveedor; registrar una compra; comprobar el aumento de existencias; realizar una venta; emitir su comprobante; consultar stock, alerta y cierre. Volver a abrir la aplicación y recuperar la venta y el saldo correspondiente.

Ampliaciones opcionales. Lectura de códigos de barras, promociones, varios medios de pago en una venta o exportación CSV.

3. Investigación

Acérquense a un kiosco, almacén o pequeño comercio. Conversen con quien atiende la caja, recibe mercadería y decide las reposiciones; una persona puede desempeñar varias de esas funciones.

Qué observar. Sigan una recepción de mercadería y una venta: identificación del producto, carga de cantidades, elección del medio de pago y consulta del saldo. Observen cómo comparan lo recibido con el comprobante y cómo detectan faltantes al cerrar la jornada.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué inicia una compra al proveedor y qué comprobación permite dar por recibida la mercadería?
- ¿Cómo distinguen productos parecidos y qué datos registran de cada presentación y proveedor?
- ¿En qué momento descuentan las unidades vendidas y qué hacen si el saldo registrado no coincide con el disponible?
- ¿Qué ocurre si cambia un precio mientras preparan una venta o si cargan una cantidad incorrecta?
- ¿Cómo deciden cuándo reponer y qué productos necesitan revisar al comenzar el día?
- ¿Cómo corrigen o anulan una venta y cómo evitan devolver dos veces las mismas unidades al stock?
- ¿Qué operaciones y medios de pago comparan en el cierre y cómo revisan una diferencia?

Materiales que conviene conocer. Soliciten conocer, sin datos comerciales sensibles, una ficha de producto, un remito de recepción, un comprobante interno de venta y una planilla de cierre. Registren qué campos utiliza realmente cada persona.

Del hallazgo al requisito. Relacionen un problema observado con una regla verificable: si una venta se confirma con saldo insuficiente, formulen el requisito de rechazarla sin registrar comprobante ni descontar unidades. Describan los casos de uso Registrar compra, Confirmar venta y Consultar cierre, con sus alternativas.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si

no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

El DER (diagrama entidad-relación) representa entidades, atributos y vínculos; la cardinalidad indica cuántos registros participan. Una clave primaria identifica un registro y una foránea lo relaciona con otro. Normalizar busca evitar duplicaciones que provoquen contradicciones.

Entidades y relaciones orientativas. Producto puede pertenecer a Categoría. Compra se vincula con Proveedor y sus detalles con productos. Venta contiene detalles y un medio de pago. MovimientoStock permite relacionar cada variación con la operación que la originó. Deberán decidir cómo representar existencias y evitar que un saldo almacenado difiera de los movimientos.

Del análisis al modelo. Partan del recorrido compra–venta para listar identificador, descripción, estado, mínimo de reposición, cantidad y precio aplicado. En el DER indiquen, por ejemplo, una categoría con varios productos y una venta con varios detalles; cada detalle referencia un producto. Traduzcan estas relaciones con claves primarias y foráneas; definan campos obligatorios, identificadores únicos y restricciones de cantidades e importes. Eviten repetir los datos del proveedor en cada detalle y justifiquen la normalización.

Maestros, operaciones e historial. Distingan catálogos actualizables de compras, ventas y movimientos históricos. ¿Qué ocurre al desactivar un producto con ventas? ¿Cómo se conserva el comprobante si cambia la descripción o el precio? Documenten una estrategia coherente entre saldo y movimientos, y qué registro permite reconocer una anulación ya aplicada.

Implementación e integración. Preparen scripts SQL de creación y carga ficticia con un producto sin stock, otro en el mínimo y otro disponible. Integren CRUD de catálogos desde Qt y prueben restricciones sobre registros relacionados. La transacción de venta debe coordinar cabecera, detalles y stock; documenten una prueba en la que un rechazo no deja una venta parcial.

Consultas orientativas y persistencia aplicable. Existencias y productos a reponer; ventas por fecha, categoría o medio de pago; compras por proveedor; detalle de un comprobante y cierre diario.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Confirmar venta y Registrar compra antes de programarlos. Incluyan producto inactivo, cantidad inválida, stock insuficiente, cálculo del total, fallo de escritura y confirmación. Representen también cómo se obtiene el cierre sin volver a afectar existencias.

Clases y responsabilidades. Asignen a las entidades la protección de sus valores y a un servicio la coordinación del comprobante. Comparen composición entre venta y detalles con asociación a producto y medio de pago; un detalle no debe depender de un widget. Mantengan el diagrama de clases alineado con esos acuerdos.

Aplicación técnica orientativa. Venta y DetalleVenta permiten trabajar composición; un servicio valida y calcula el total. Un std::vector puede reunir ítems y un algoritmo de búsqueda localizar un producto. Qt puede mostrar catálogo, carrito y confirmación. SQL permite relacionar comprobantes y productos, agregar ventas y detectar mínimos de stock.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Cantidades positivas, importes no negativos y productos activos. La venta no puede dejar stock negativo. Cabecera, detalles y actualización de existencias se confirman juntos. El comprobante conserva cantidades y precios de la operación aunque después cambie el precio del producto. Definir un mínimo por producto y alertar cuando el saldo sea igual o inferior.
- El cierre agrupa ventas válidas de una fecha por medio de pago; no vuelve a descontar stock. Para evitar ambigüedad, el cierre académico es una consulta consolidada que puede regenerarse. Si se permite anular ventas, deberá registrarse la anulación y su reversión de stock una sola vez.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con la venta de un producto disponible, invocada sin la pantalla definitiva; verifiquen el total, persistan sus efectos y recién entonces integren el carrito. Amplíen a varios ítems, compra, reposición y cierre. Prueben cantidad cero, venta por todo el stock y una unidad por encima del saldo.

Evidencias de funcionamiento que deben mostrar. Mostrar stock antes y después de comprar y vender, rechazar una venta sin existencias, consultar el comprobante persistido y verificar que el cierre coincide con las operaciones registradas. La baja de un producto con movimientos no deberá borrar su historial.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Como propuesta inicial, boceten catálogo de productos, recepción de compra, carrito de venta y consulta de stock/cierre. Tracen la navegación desde QMainWindow hacia el alta de producto y la confirmación de venta, y cómo se regresa al comprobante guardado.

Qué representar en los bocetos. En el carrito señalen producto, cantidad, precio aplicado, subtotal, total y medio de pago. Dibujen también carrito vacío, producto inactivo, falta de stock y error al guardar. Indiquen qué acción solicita confirmación y qué datos deben permanecer para corregir un rechazo sin duplicar la venta.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de Confirmar envía los ítems al servicio de venta; el backend comprueba y calcula, y el acceso a datos confirma. Actualicen comprobante y existencias únicamente con la respuesta exitosa. Consultar cierre debe invocar una consulta, sin ejecutar nuevamente el cobro.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-kiosco» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Confirmar venta sin stock negativo», finalizada cuando una venta válida persiste y otra excesiva deja el saldo intacto; «Cierre por medio de pago», finalizada cuando los totales coinciden con comprobantes ficticios conocidos.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Valida stock al confirmar detalles de venta» / «Documenta prueba de cierre por medio de pago»

Revisión integrada. Integren primero producto, servicio de venta y una pantalla mínima; revisen juntos saldo, comprobante y consulta tras reiniciar. Una modificación de precios o movimientos debe incluir la revisión de scripts, pruebas y manual.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan cómo se controlan hoy existencias y ventas, qué errores de registro buscan reducir y qué necesitan caja y administración. Justifiquen la viabilidad de un local y una caja; el beneficio esperado puede ser conocer reposiciones y reconstruir ventas, sin prometer resultados medidos que no

investigaron. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.

- **Investigación y requisitos.** Documenten actores y pasos de compra, venta y cierre, con alternativas por falta de stock y, si implementan anulación, por esa operación. Vinculen cada hallazgo con el requisito que lo atiende y con una evidencia de funcionamiento. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Expliquen la conservación de precios, el cálculo del saldo, los límites de la transacción y la naturaleza consultiva del cierre. Justifiquen cómo una baja conserva movimientos y, si admiten anulaciones, cómo evitan revertirlas dos veces. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Guíen el acceso, el alta de producto y proveedor, la compra que incrementa stock, la venta con comprobante y la consulta de reposición y cierre. Incluyan capturas con datos ficticios, rechazo por falta de unidades y recuperación de la venta después de reabrir.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Planifiquen primero catálogo y compra, luego venta integrada y finalmente cierre; registren la decisión sobre saldos y el criterio para dar de baja productos. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Usen un saldo inicial conocido y registren cantidades e importes antes y después; comparen el cierre con ventas válidas y, si implementan anulación, con su reversión única. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.2 Cine

1. Nombre del tema

Cine. Sistema de programación de funciones y boletería de cine, con mapa de butacas, bloqueos, reservas, ventas y consulta de ocupación.

2. Objetivo

Gestionar la programación y comercialización de entradas de un cine pequeño. El administrador configura películas, salas y precios; la boletería administra reservas y ventas.

Alcance obligatorio y límites. Administrar películas, salas y funciones; definir precios; mostrar un mapa de butacas; bloquear temporalmente asientos, reservar, vender y anular entradas; consultar ocupación. La venta y la reserva son locales. No se exige pago por Internet. Una reserva conserva la referencia del cliente y un vencimiento, y puede convertirse en venta.

Recorrido principal obligatorio. Crear una función, seleccionar una butaca libre, obtener un bloqueo, registrar una reserva y convertirla en venta antes de su vencimiento. Consultar la entrada, anularla y verificar la liberación según las reglas. Mostrar también el vencimiento de un bloqueo sin confirmación.

Ampliaciones opcionales. Selección automática de asientos contiguos, descuentos o varias boleterías conectadas.

3. Investigación

Consulten una boletería de cine, un espacio cultural que proyecte películas o a quien organice funciones. Conversen con personal de venta y programación; si solo pueden observar un sistema público, describan esa limitación sin atribuirle procedimientos internos no conocidos.

Qué observar. Observen la selección de función y butaca, el tiempo entre selección y confirmación, la identificación de reservas y la comunicación de disponibilidad. Revisen cómo distinguen una selección temporal de una entrada ya vendida.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué datos necesitan para publicar una función y cómo verifican que la sala no esté ocupada por otra proyección?
- ¿Qué sucede con una butaca desde que se selecciona hasta que se confirma la operación?
- ¿Cuánto dura una reserva, cómo identifican a quien la hizo y qué ocurre si llega después del vencimiento?
- ¿Cómo convierten una reserva en entrada sin ocupar dos veces la misma butaca?
- ¿Qué hacen si dos intentos de venta solicitan el mismo asiento o si la confirmación falla?
- ¿Hasta cuándo anulan entradas y qué información conservan del comprobante anulado?
- ¿Cómo calculan ocupación y qué diferencia hacen entre butacas vendidas, reservadas y bloqueadas?

Materiales que conviene conocer. Revisen un plano de sala, una programación, un comprobante de reserva, una entrada y un reporte de ocupación. Pidan ejemplos vacíos o anonimizados y anoten la información que cambia según la función.

Del hallazgo al requisito. Transformen la observación de reservas vencidas en un requisito de disponibilidad recalculada antes de confirmar. Escriban los casos Seleccionar butaca, Reservar, Convertir en venta y Anular, indicando quién puede actuar sobre cada bloqueo y qué ocurre al vencer el plazo.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si

no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Sala contiene Butacas y Función relaciona sala, película, fecha y hora. La ocupación corresponde a la combinación Función y Butaca, no a la butaca aislada. Entrada, Reserva y Bloqueo pueden representarse mediante entidades o estados relacionados; el equipo deberá justificar su decisión.

Del análisis al modelo. Identifiquen sala, fila, número de butaca, función, inicio, duración, precio, titular y vencimiento. Expresen cardinalidades: una sala contiene butacas y recibe funciones en horarios compatibles; una asignación une una función con una butaca. Definan claves primarias, foráneas y una protección de unicidad vigente. Eviten duplicar el plano físico por cada entrada y justifiquen cómo normalizan salas y películas.

Maestros, operaciones e historial. Separen películas y salas de funciones, asignaciones y entradas históricas. ¿Se puede cambiar el plano de una sala con entradas emitidas? ¿Qué sucede al cancelar una reserva o desactivar una película programada? Preserven referencias y precios aplicados; no eliminen una entrada para simular su anulación.

Implementación e integración. Creen scripts SQL y datos ficticios con dos funciones, butacas libres y ocupadas, bloqueos vencidos y reservas vigentes. Integren CRUD de programación y consultas de ocupación. Coordinen comprobación de disponibilidad y cambio de asignación en una transacción; documenten campos obligatorios, estados válidos y el mecanismo que impide doble venta.

Consultas orientativas y persistencia aplicable. Ocupación por función o película, ventas y anulaciones por fecha, reservas por vencer. Diferenciar entradas vendidas de reservas o bloqueos para calcular porcentajes.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Representen bloquear–reservar–vender y el vencimiento del bloqueo. Incluyan titular incorrecto, función iniciada, reserva vencida, asiento ocupado y fallo al guardar. Otro flujo debe mostrar anulación antes del inicio y conservación del registro.

Clases y responsabilidades. Modelen estados con operaciones significativas, como reservar o confirmar venta, evitando que la pantalla asigne libremente el estado. Un servicio puede coordinar Función y asignaciones; diferencien composición de sala y butacas de las asociaciones históricas con entradas.

Aplicación técnica orientativa. Un servicio de reservas coordina estados; std::vector representa butacas y algoritmos STL filtran disponibles. Qt permite una grilla de asientos diferenciados por estado y un temporizador visual. SQL consulta ocupaciones vigentes y conserva entradas sin duplicar asignaciones.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Documentar una duración de bloqueo y un vencimiento de reserva. El bloqueo conserva función, asiento, vencimiento y operación o sesión titular. Vencido el plazo, pierde vigencia: la disponibilidad se recalcula al consultar y antes de confirmar, aunque un temporizador actualice la pantalla. La reserva no vendida vence como máximo al iniciar la función; la venta solo procede antes de su inicio.
- Una transacción debe verificar disponibilidad y registrar el cambio; una clave única para la asignación vigente por función y butaca, o un mecanismo equivalente, debe impedir la doble venta. Anular antes del inicio libera el asiento y conserva la entrada anulada. También deberán impedir funciones superpuestas en la misma sala según duración y horario de la película.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Construyan primero la venta de una butaca de una función futura; ensayen un segundo intento sobre ella. Agreguen bloqueo y vencimientos, luego conversión de reserva y anulación. Prueben el instante del vencimiento y el inicio de función, no solo plazos claramente válidos.

Evidencias de funcionamiento que deben mostrar. Dos intentos sucesivos sobre el mismo asiento no producen dos ventas; un bloqueo vencido libera disponibilidad; una reserva vendida no queda duplicada; una anulación queda registrada; al reiniciar se conserva la ocupación correcta.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Bocetos sugeridos: programación, edición de función, mapa de sala, confirmación de reserva/venta e historial de entradas. Dibujen el paso de función seleccionada a mapa y de mapa a comprobante, con retorno que consulte disponibilidad actualizada.

Qué representar en los bocetos. Marquen película, sala, fecha, precio, leyenda de estados, asiento elegido y tiempo restante. Prevean función sin butacas disponibles, bloqueo vencido mientras la ventana sigue abierta y reserva que no puede convertirse. La leyenda y los mensajes deben seguir siendo claros en ambos temas, sin depender únicamente del color.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El temporizador visual solo informa: el servicio vuelve a validar vencimiento, titular y disponibilidad al confirmar. La interfaz recibe el estado persistido de la asignación y actualiza el mapa; no decide que una entrada está vendida solo porque se coloreó un botón.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-cine» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Liberar bloqueos vencidos», terminada al recuperar disponibilidad al consultar y al confirmar; «Convertir reserva en venta», terminada cuando existe una sola asignación vigente y el comprobante puede recuperarse.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Revalida vencimiento antes de vender una butaca» / «Agrega prueba de anulación previa a la función»

Revisión integrada. Revisen juntos mapa, estado en la base y resultado del servicio sobre la misma función. Integren la regla de vencimiento antes de completar el temporizador visual y comprueben la ocupación tras reiniciar.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Presenten las dificultades de coordinar disponibilidad y reservas y los usuarios de programación y boletería. Limiten la solución a venta local y comprobantes internos; expliquen por qué evitar asignaciones duplicadas es un beneficio esperado del proyecto. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Describan una reserva que se vende, otra que vence y una entrada anulada, con actores, plazos, alternativas y resultados. Distingan hechos relevados de duraciones académicas definidas por el equipo. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen la identidad función–butaca, la protección de la asignación vigente, el tratamiento de vencimientos y la diferencia entre bloqueo, reserva y venta. Documenten cómo impiden funciones

superpuestas y qué se conserva al anular. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Expliquen login, creación de función, selección de butaca, bloqueo, reserva, conversión y consulta de entrada. Muestren vencimiento y anulación con mensajes y capturas ficticias, y cómo verificar la ocupación al volver a abrir.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Relacionen programación, estados de asignación y mapa con tareas coordinadas; registren como decisiones las duraciones y la política de cambios sobre salas ya utilizadas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Preparen una función futura y un asiento identificable; registren dos intentos de venta, vencimiento, conversión sin duplicación y anulación con registro conservado. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.3 Biblioteca

1. Nombre del tema

Biblioteca. Sistema de catálogo y circulación de biblioteca, con copias identificables, préstamos, renovaciones, reservas, devoluciones y multas.

2. Objetivo

Organizar el catálogo y la circulación de una biblioteca. El personal administra obras y socios, realiza préstamos y controla devoluciones, reservas y deudas.

Alcance obligatorio y límites. Administrar catálogo, autores, ejemplares y socios; registrar préstamos, devoluciones y renovaciones; gestionar reservas, multas y disponibilidad por ejemplar. El catálogo identifica una obra o edición; cada ejemplar representa una copia física con identificador propio. El sistema debe poder prestar una copia y mantener disponibles las restantes.

Recorrido principal obligatorio. Registrar obra, autores, copias y socio; prestar una copia; mostrar el vencimiento y la disponibilidad de las demás; intentar una renovación; registrar una reserva de otro socio; devolver el ejemplar y comprobar multa, estado y prioridad de reserva cuando correspondan.

Ampliaciones opcionales. Lectura de etiquetas, importación de catálogo o avisos de vencimiento por medios externos.

3. Investigación

Visiten una biblioteca escolar, universitaria o popular y conversen con personal de catálogo y préstamos. Si consultan lectores, distingan su experiencia de las reglas que confirma la persona encargada de circulación.

Qué observar. Sigan cómo localizan una obra, identifican la copia entregada, comprueban la situación del socio y registran vencimientos. Observen la devolución y cómo separan un ejemplar reservado de otro disponible.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Cómo distinguen una obra o edición de cada copia física y qué identificador anotan al prestarla?
- ¿Qué controles realizan sobre el socio y sus préstamos antes de entregar un ejemplar?
- ¿Cómo fijan vencimientos y cuántos préstamos puede mantener una persona al mismo tiempo?
- ¿En qué circunstancias permiten renovar y qué cambia cuando otro socio espera la misma obra?
- ¿Cómo ordenan reservas, avisan disponibilidad y resuelven una reserva que no se retira?
- ¿Cómo registran una devolución tardía, calculan la multa y dejan constancia de que fue saldada?
- ¿Qué consultan para encontrar vencidos o reconstruir el historial sin confundir copias de la misma obra?

Materiales que conviene conocer. Revisen ficha bibliográfica, etiqueta de ejemplar, ficha de socio, comprobante de préstamo, listado de vencidos y registro de reservas. Utilicen formularios vacíos o datos anonimizados con autorización.

Del hallazgo al requisito. Si una obra tiene copias con disponibilidad distinta, traduzcan el hallazgo en requisitos por ejemplar. Elaboren casos Prestar, Renovar, Reservar y Devolver, con rechazo por deuda y alternativas por demora o reserva prioritaria; no sustituyan las reglas académicas por políticas de la biblioteca visitada.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Obra puede relacionarse con varios Autores y tener varios Ejemplares. Préstamo vincula un Socio con un Ejemplar. Reserva puede referirse a una obra para esperar la próxima copia disponible. Multa se asocia al préstamo que originó la demora. No deberán confundir la descripción bibliográfica con la identidad de una copia.

Del análisis al modelo. Identifiquen datos bibliográficos, código de ejemplar, estado del socio y fechas de préstamo, vencimiento y devolución. Representen obra–ejemplar como uno a muchos y analicen obra–autor como muchos a muchos. Transformen el DER con claves primarias y foráneas, campos obligatorios y código de copia único. Normalicen autores y obras sin repetir sus descripciones en cada préstamo.

Maestros, operaciones e historial. El catálogo y los socios son datos maestros; préstamos, renovaciones, reservas y multas explican la circulación. ¿Qué ocurre con préstamos abiertos al desactivar un socio? ¿Puede retirarse del catálogo una copia con historial? Decidan cómo conservar fechas anteriores de una renovación y la referencia del préstamo que origina una multa.

Implementación e integración. Generen scripts SQL y datos ficticios con varias copias de una obra, socios con y sin deuda y reservas ordenadas. Integren CRUD del catálogo y socios con las consultas de circulación. Al devolver, coordinen estado del préstamo, multa única y destino del ejemplar en una transacción; prueben que una referencia inválida o un rechazo no produce una devolución parcial.

Consultas orientativas y persistencia aplicable. Disponibilidad por obra y copia, préstamos vencidos, historial del socio, reservas pendientes y multas sin saldar.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Otorgar préstamo, Renovar y Devolver. Incluyan disponibilidad, socio activo, deuda, límite de préstamos, vencimiento y reserva de otro socio. En la devolución muestren la decisión entre disponibilidad y apartado, y entre multa cero y multa por demora.

Clases y responsabilidades. Diferencien Obra de Ejemplar en clases y responsabilidades. Un servicio de circulación puede coordinar asociaciones con Socio y Préstamo; la copia protege sus cambios de estado. Mantengan la prioridad de reservas fuera del formulario de devolución.

Aplicación técnica orientativa. Ejemplar encapsula transiciones de estado y un servicio coordina el préstamo. Una cola o colección ordenada puede representar reservas; std::find_if permite localizar préstamos. Qt muestra catálogo y circulación. SQL relaciona obras, autores, copias, socios y préstamos.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Definir y documentar plazo de préstamo, límite de préstamos activos por socio, extensión por renovación, valor diario de multa y plazo para retirar una reserva. Un ejemplar no puede tener dos préstamos activos. Solo un socio activo puede solicitar préstamo; se bloquearán nuevos préstamos mientras tenga multas pendientes. La renovación se admite antes del vencimiento si no hay reserva pendiente de otro socio.
- Las reservas se atienden por orden de registro, sin duplicados activos del mismo socio y obra. La devolución calcula la demora en días completos posteriores al vencimiento; si no hay demora, la multa es cero. La multa se registra una sola vez y puede marcarse como saldada con importe y fecha. El ejemplar devuelto queda disponible o apartado para la siguiente reserva, según corresponda.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Prueben primero el préstamo de una copia dejando otra disponible. Incorporen devolución, renovación y reserva en ese orden de integración. Usen como límites el máximo de préstamos del socio, el vencimiento exacto y una devolución sin días completos de demora.

Evidencias de funcionamiento que deben mostrar. Prestar una copia sin bloquear toda la obra; rechazar un segundo préstamo de esa copia; impedir una renovación incompatible; devolver con y sin demora; comprobar que repetir la consulta no vuelve a generar una multa y que el historial persiste.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Propongan catálogo con detalle de copias, ficha del socio, préstamo/devolución y consultas de reservas y multas. Dibujen cómo se llega desde una obra a un ejemplar y desde el socio a su historial; no representen disponibilidad con un único indicador para toda la obra.

Qué representar en los bocetos. Señalen código de copia, socio, fechas, situación de deuda y motivo de rechazo. Boceten obra sin copias disponibles, reserva pendiente de otro socio y devolución que aparta el ejemplar. La confirmación debe identificar la copia concreta y el resultado de la operación.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de Devolver solicita el caso de uso; el servicio calcula demora y resuelve reserva, y la base confirma los cambios. Qt muestra el importe y el nuevo estado sin calcular otra multa al refrescar la ficha ni permitir modificar directamente una devolución cerrada.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-biblioteca» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Préstamo por ejemplar», terminada cuando prestar una copia deja libre otra de la misma obra; «Devolución con reserva», terminada cuando se respeta el orden de espera y no se duplica la multa.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Impide renovar con reserva de otro socio» / «Conserva multa única al registrar devolución»

Revisión integrada. Integren catálogo, circulación y consulta del socio con un conjunto común de copias. Revisen que modelo, interfaz y servicio utilicen el mismo identificador de ejemplar y que el historial sobreviva a bajas lógicas.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Expliquen la necesidad de controlar circulación, disponibilidad y vencimientos para bibliotecarios y lectores. Delimiten una biblioteca local y justifiquen que la solución académica puede seguir cada copia y conservar la relación entre préstamo, reserva y multa. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten hallazgos sobre copias, plazos y espera; escriban casos de uso con alternativas por deuda, renovación incompatible y devolución tardía. Identifiquen qué parámetros fija el equipo y cómo se aplican de manera uniforme. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen obra frente a ejemplar, relaciones con autores, orden de reservas, cálculo de días completos y registro único de multa. Expliquen qué operaciones modifican varios registros y cómo conservan historial ante bajas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Muestren acceso, alta de obra/autores/copias y socio, préstamo, renovación, reserva y devolución. Incluyan consulta y saldo de multa, disponibilidad de las demás copias y recuperación del historial con capturas ficticias.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen catálogo y circulación desde el primer avance; registren la decisión sobre plazos, límite de préstamos y vencimiento de reservas junto con las tareas que implementan esas reglas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Documenten préstamo duplicado de una copia, renovación bloqueada, devolución puntual y tardía, prioridad de reserva y consulta reiterada sin nuevas multas. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.4 Agencia de transporte

1. Nombre del tema

Agencia de transporte. Sistema local de venta y cancelación de pasajes para salidas programadas, con asientos identificados y manifiesto de pasajeros.

2. Objetivo

Administrar pasajes de una agencia con salidas programadas. El administrador configura recorridos y unidades; la persona operadora vende, cancela y consulta el manifiesto.

Alcance obligatorio y límites. Administrar destinos, recorridos, unidades, horarios, tarifas y pasajeros; programar salidas; asignar asientos; emitir y cancelar pasajes; generar un manifiesto de viaje. Para el alcance académico, cada recorrido comprende un origen y un destino y cada pasaje ocupa su asiento durante todo el viaje. La venta por tramos intermedios queda fuera del alcance obligatorio.

Recorrido principal obligatorio. Crear recorrido, unidad y salida; consultar disponibilidad; registrar pasajero; seleccionar asiento y emitir el pasaje; recuperar el manifiesto; cancelar el pasaje antes de la salida y verificar que el asiento vuelve a estar disponible.

Ampliaciones opcionales. Recorridos con tramos, cambios de pasaje, descuentos o impresión de comprobantes con códigos de verificación.

3. Investigación

Consulten una agencia de pasajes o empresa de transporte y conversen con personal de venta, programación y despacho. Centren el relevamiento en recorridos completos de origen a destino para mantener el alcance académico.

Qué observar. Observen la búsqueda de salidas, selección de asiento, identificación del pasajero y emisión del comprobante. Revisen cómo preparan el manifiesto y comunican un cambio de unidad o una cancelación.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué información confirma una salida y cómo relacionan recorrido, unidad, horario y llegada prevista?
- ¿Cómo identifican los asientos de cada unidad y cómo consultan cuáles siguen disponibles?
- ¿Qué datos del pasajero y de la tarifa quedan registrados al emitir un pasaje?
- ¿Qué hacen si dos operaciones solicitan el mismo asiento o si falla la emisión?
- ¿Hasta qué momento cancelan un pasaje y cómo conservan el registro al liberar el lugar?
- ¿Cómo revisan un reemplazo de unidad cuando ya hay asientos vendidos?
- ¿Qué datos necesita el personal de despacho en el manifiesto y cómo distingue pasajes vigentes de cancelados?

Materiales que conviene conocer. Revisen un horario de salidas, un esquema de asientos, un pasaje, una constancia de cancelación y un manifiesto anonimizado. Registren qué información pertenece a la unidad y cuál a una salida concreta.

Del hallazgo al requisito. Conviertan el riesgo de asignación repetida en un requisito por salida y asiento, no por número de asiento global. Escriban Emitir pasaje, Cancelar y Consultar manifiesto con actor, disponibilidad inicial, confirmación y alternativas por viaje iniciado o unidad incompatible.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Recorrido une destinos; Salida concreta vincula recorrido, unidad, fecha y horario. Unidad tiene capacidad y asientos identificables. Pasaje relaciona pasajero, salida y asiento, con tarifa aplicada y estado. El mismo número de asiento puede utilizarse en salidas distintas.

Del análisis al modelo. Identifiquen códigos de destino y unidad, capacidad, asientos, salida, llegada prevista, pasajero, tarifa y estado del pasaje. En el DER separen un recorrido de sus múltiples salidas y una unidad de sus viajes. Usen claves primarias y foráneas, campos obligatorios y unicidad de asignación vigente por salida/asiento. Normalicen destinos y unidades sin copiar toda su información en cada pasaje.

Maestros, operaciones e historial. Distingan tarifas y recorridos maestros de pasajes emitidos con su precio histórico. ¿Puede eliminarse un pasajero con viajes registrados? ¿Qué se conserva al cancelar? ¿Qué cambios de horario o unidad son compatibles con ventas vigentes? Justifiquen restricciones de modificación y bajas que no rompan el manifiesto histórico.

Implementación e integración. Preparen scripts SQL con dos salidas que compartan números de asiento, una salida completa y pasajes cancelados. Integren CRUD y filtros por destino/fecha. La emisión debe comprobar y guardar la asignación de forma consistente; la cancelación debe conservar el pasaje y liberar solo su lugar, sin afectar otra salida.

Consultas orientativas y persistencia aplicable. Manifiesto con salida, unidad, pasajero y asiento; ocupación por viaje; pasajes emitidos y cancelados; ingresos internos por recorrido o período.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Representen emisión y cancelación y, si permiten reemplazar una unidad, la validación de ese cambio. Incluyan asiento inexistente, capacidad agotada, salida iniciada, unidad ocupada en otro viaje y fallo de escritura. Marquen el punto en que se confirma el pasaje y queda disponible para el manifiesto.

Clases y responsabilidades. Asignen a Salida la información de su viaje y a Pasaje su estado y tarifa histórica. Un servicio coordina disponibilidad y emisión; asocien pasajero y unidad sin que sus formularios contengan las reglas. Mantengan el diagrama de clases consistente con los estados realmente utilizados.

Aplicación técnica orientativa. Salida y Pasaje encapsulan capacidad y estado, coordinados por un servicio de emisión. std::vector reúne asientos y un algoritmo STL filtra salidas. Qt permite buscar por destino y fecha y seleccionar lugares. SQL obtiene pasajeros por salida y protege la asignación única vigente.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- La capacidad corresponde a la unidad asignada a esa salida. No se admite un asiento inexistente ni dos pasajes activos para la misma salida y asiento. La emisión verifica disponibilidad y guarda la asignación en una operación consistente. Conservar la tarifa de emisión aunque cambien precios futuros. Una cancelación anterior a la salida libera el lugar, conserva el registro y solo puede aplicarse una vez.
- No vender ni cancelar una salida ya iniciada en este modelo. Definir duración o llegada prevista para evitar asignar la misma unidad a viajes superpuestos. Si la unidad se reemplaza, el sistema deberá comprobar que la nueva disposición admite todos los asientos vendidos; en caso contrario, rechazar el cambio con un mensaje claro.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Implementen primero una salida con pocos asientos ficticios y emitan un pasaje. Comprueben una segunda asignación del mismo lugar, luego cancelación y reutilización. Amplíen a varias salidas y prueben la última plaza, el instante de inicio y un reemplazo con menos asientos.

Evidencias de funcionamiento que deben mostrar. Completar una salida hasta su capacidad, rechazar una asignación duplicada, cancelar y reutilizar un asiento sin perder historial, recuperar el manifiesto al reiniciar y comprobar que una venta no modifica otra salida.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten búsqueda por destino/fecha, administración de salidas, selección de asiento, datos del pasajero y manifiesto. Dibujen una navegación que mantenga visible la salida elegida al abrir el diálogo de emisión o cancelación.

Qué representar en los bocetos. Muestren unidad, horario, asiento, tarifa aplicada y estado. Prevean lista de salidas vacía, unidad completa, asiento ocupado al confirmar y viaje ya iniciado. El comprobante de cancelación debe referenciar el pasaje conservado y el manifiesto debe distinguir vigentes de anulados.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. La selección visual se convierte en una solicitud al servicio de emisión. Backend verifica capacidad y estado de la salida; la capa de datos confirma la asignación. El slot actualiza el mapa y permite consultar el manifiesto sin reconstruirlo a partir de botones coloreados.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-transporte» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Asignación única por salida», terminada cuando el mismo número funciona en viajes diferentes y se rechaza su duplicación en uno; «Cambio de unidad compatible», terminada al aceptar solo disposiciones que admiten todos los pasajes vendidos.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Valida asientos vendidos al reemplazar unidad» / «Agrega manifiesto filtrado por salida»

Revisión integrada. Integren búsqueda, emisión y manifiesto con las mismas salidas ficticias. Revisen conjuntamente los efectos de cancelar y reiniciar para detectar consultas que mezclen asientos de viajes distintos.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan la necesidad de coordinar disponibilidad, venta y despacho para una agencia. Justifiquen el alcance por viaje completo y los beneficios esperados de un manifiesto confiable, sin incluir venta por tramos como obligación. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Relacionen hallazgos con casos de programación, emisión, cancelación y consulta. Documenten actores, datos requeridos, alternativas por capacidad y límites temporales; distingan la política observada de la regla académica de no operar viajes iniciados. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Expliquen unicidad salida–asiento, tarifa histórica, capacidad de la unidad, intervalos de viaje y validación del reemplazo. Justifiquen qué datos integran el manifiesto y qué se mantiene tras cancelar o desactivar registros maestros. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Describan login, alta de recorrido/unidad/salida, búsqueda, registro del pasajero, elección de asiento, emisión y manifiesto. Muestren cancelación anterior al viaje, reutilización del asiento y consulta posterior con datos ficticios.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Relacionen diseño del mapa con capacidad y numeración de unidades; registren decisiones sobre duración, llegada prevista y cambios permitidos antes de programar emisión y cancelación. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Preparen capacidad completa, asiento inexistente, doble asignación, cancelación única y otra salida que no debe cambiar; documenten también un reemplazo de unidad rechazado. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.5 Complejo deportivo

1. Nombre del tema

Complejo deportivo. Sistema de agenda y cobros de un complejo deportivo, con reservas por cancha, cancelaciones y períodos de mantenimiento.

2. Objetivo

Organizar el uso de canchas de un complejo y evitar compromisos incompatibles. El personal administra recursos, clientes, reservas, pagos y períodos de mantenimiento.

Alcance obligatorio y límites. Administrar deportes, canchas y franjas horarias; registrar clientes; reservar y cancelar turnos; registrar pagos y bloquear canchas por mantenimiento. El alcance comprende una sede, reservas individuales de una cancha y registro interno de cobros. Deberá existir una agenda consultable por fecha y recurso.

Recorrido principal obligatorio. Configurar una cancha y sus horarios; consultar disponibilidad; registrar cliente y reserva; registrar su pago; intentar un turno superpuesto; cancelar la reserva y verificar disponibilidad. Registrar un mantenimiento y comprobar el bloqueo de ese período.

Ampliaciones opcionales. Abonos, reservas recurrentes, lista de espera o recordatorios externos.

3. Investigación

Acérquense a un complejo de canchas o club y conversen con recepción, administración y personal de mantenimiento. Investiguen cómo coordinan la disponibilidad deportiva con bloqueos del recurso.

Qué observar. Sigan una solicitud de turno desde la consulta de horarios hasta la reserva y el pago. Observen cómo cambian la agenda cuando se cancela y cómo detectan que una reparación afecta turnos ya comprometidos.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué datos solicitan para reservar una cancha y cómo acuerdan duración y horario?
- ¿Cómo determinan qué deportes admite cada cancha y qué franjas pueden ofrecer?
- ¿Cómo distinguen turnos contiguos de turnos que se superponen?
- ¿Cómo registran una seña o pago y qué información permite conocer el saldo de la reserva?
- ¿Qué hacen al cancelar una reserva ya cobrada y cómo documentan una devolución?
- ¿Cómo proceden si un mantenimiento necesario coincide con una reserva confirmada?
- ¿Qué consultas necesitan para comparar ocupación, cancelaciones y tiempo bloqueado por mantenimiento?

Materiales que conviene conocer. Soliciten ver una agenda, ficha de cliente, reserva, recibo interno y registro de mantenimiento. Comparen qué estados se muestran al público y cuáles necesita administración, usando ejemplos sin datos personales.

Del hallazgo al requisito. A partir de conflictos de agenda, definan Reservar cancha y Bloquear mantenimiento con alternativas explícitas. Conviertan la atención de turnos contiguos en una prueba con fin igual a inicio; el hallazgo ayuda a diseñar, pero no permite omitir pagos o mantenimiento.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Cancha se vincula con un deporte o con los deportes admitidos. Reserva relaciona cliente, cancha e intervalo; Pago se asocia con una reserva. Mantenimiento bloquea una cancha en un período. Las franjas disponibles se derivan del horario de atención y de las ocupaciones vigentes.

Del análisis al modelo. Identifiquen cancha, deportes admitidos, horario de atención, cliente, inicio, fin, importe y movimientos de cobro. Expresen relaciones entre cancha, reserva y cliente; evalúen si cancha–deporte requiere una relación muchos a muchos. Traduzcan el DER con claves primarias/foráneas, campos obligatorios y valores válidos. Normalicen datos de clientes y deportes, sin repetirlos como texto en toda la agenda.

Maestros, operaciones e historial. Separen canchas y tarifas maestras de reservas con precio pactado, pagos, devoluciones y mantenimientos. ¿Qué ocurre al desactivar una cancha con turnos futuros? ¿Cómo se anula una reserva cobrada sin borrar el pago? ¿Puede editarse un mantenimiento que dejaría reservas incompatibles? Documenten cada política y su efecto histórico.

Implementación e integración. Creen scripts SQL y datos de prueba con turnos contiguos, un conflicto y un mantenimiento. Integren CRUD, agenda y saldos. Coordinen comprobación de disponibilidad y alta de reserva en una transacción cuando corresponda; documenten cómo evitan dobles pagos y conservan devoluciones como movimientos asociados.

Consultas orientativas y persistencia aplicable. Agenda por cancha, ocupación por deporte, reservas canceladas, saldos pendientes y mantenimientos programados. Distinguir tiempo reservado de tiempo no disponible por mantenimiento.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Reservar, Cancelar con pago y Registrar mantenimiento. Incluyan horario fuera de franja, duración inválida, reserva superpuesta, mantenimiento existente y fallo de escritura. En el mantenimiento nuevo, muestren el rechazo cuando hay reservas y el paso previo para resolverlas.

Clases y responsabilidades. Un servicio de agenda puede evaluar intervalos; Reserva conserva estado e importe y se asocia con Cliente y Cancha. Expresen la diferencia entre ocupar por reserva y bloquear por mantenimiento sin duplicar la regla de superposición en pantallas distintas.

Aplicación técnica orientativa. Un servicio de agenda valida intervalos y disponibilidad; Reserva administra sus estados. Una colección STL puede ordenar turnos por hora. Qt puede combinar calendario, agenda y diálogo de reserva. SQL busca conflictos y consolida pagos por período.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Inicio anterior al fin y duración compatible con las franjas definidas. Usar intervalos con inicio incluido y fin excluido: un turno que termina a las 18 permite otro desde las 18. Hay superposición si el inicio nuevo es anterior al fin existente y el fin nuevo es posterior al inicio existente. Verificar reservas activas y mantenimientos antes de confirmar.
- Si un mantenimiento nuevo encuentra reservas, se rechazará su alta hasta resolverlas; no deberá cancelarlas silenciosamente. El importe de una reserva se conserva aunque cambie la tarifa. No duplicar cobros ni superar el total; si se admite un pago parcial, mostrar saldo. La cancelación conserva el historial y permite registrar la devolución de lo cobrado como movimiento asociado, sin borrar el pago original.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con reservar una cancha y consultar su agenda; prueben un turno que inicia exactamente al terminar otro. Agreguen mantenimiento, cobro y cancelación integrada. Verifiquen una superposición mínima, el horario límite de atención y un pago superior al saldo.

Evidencias de funcionamiento que deben mostrar. Aceptar turnos contiguos y rechazar superpuestos; impedir reservas en mantenimiento; cancelar liberando el recurso; conservar pagos y devoluciones; recuperar agenda y saldos entre ejecuciones.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Propongan agenda por fecha/cancha, alta y edición de recursos, diálogo de reserva, cuenta del cliente y mantenimiento. Tracen la navegación desde una franja disponible a confirmación y desde una reserva existente a pago o cancelación.

Qué representar en los bocetos. Marquen inicio, fin, duración, cliente, precio pactado, cobrado y saldo. Boceten agenda sin turnos, cancha fuera de servicio, conflicto al confirmar y cancelación con devolución. Diferencien visualmente reserva y mantenimiento y acompañen el estado con texto comprensible.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de reserva transmite cancha e intervalo al servicio, que valida todos los conflictos y solicita persistencia. Qt actualiza la agenda con el resultado confirmado; el arrastre o selección de una franja no debe crear por sí solo una reserva ni cancelar otra.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-complejo-deportivo» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Aceptar turnos contiguos», terminada cuando la prueba 17–18 y 18–19 funciona y una superposición se rechaza; «Cancelar con devolución», terminada cuando se libera la cancha y se conservan ambos movimientos.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Valida conflictos entre reservas y mantenimiento» / «Muestra saldo pendiente de una reserva»

Revisión integrada. Integren agenda y servicio de intervalos antes de completar reportes. Revisen una misma cancelación desde pantalla, historial y saldo; coordinen con datos cualquier cambio en estados de reserva o mantenimiento.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan los problemas de agenda y cobros de una sede, los usuarios de recepción y administración y la necesidad de conocer disponibilidad real. Justifiquen el beneficio esperado de evitar compromisos incompatibles y delimiten reservas individuales. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Escriban casos de reserva, pago, cancelación y mantenimiento con alternativas por conflicto y devolución. Relacionen hallazgos con horarios y franjas, manteniendo la regla académica de intervalos con fin excluido. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen la fórmula de superposición, la conservación de tarifas, el cálculo de saldo y la diferencia entre disponibilidad, ocupación y mantenimiento. Documenten cómo coordinan pago/devolución y cómo impiden cancelaciones silenciosas al bloquear una cancha. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Muestren acceso, configuración de cancha y franjas, consulta, reserva, pago, intento incompatible y cancelación. Incluyan alta de mantenimiento, agenda y recuperación de saldos con capturas de datos ficticios.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen agenda, regla de intervalos y pagos como tareas relacionadas; registren decisiones sobre franjas, saldos y resolución de reservas antes de un mantenimiento. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Documenten turnos contiguos y superpuestos, bloqueo por mantenimiento, pago duplicado, devolución asociada y recuperación de agenda al reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.6 Restaurante o bar

1. Nombre del tema

Restaurante o bar. Sistema de pedidos, preparación y cobro para un restaurante o bar, con división de consumos, recetas y cierre de caja.

2. Objetivo

Coordinar pedidos y cobros de un local. Administración mantiene carta y recetas; atención registra consumos y caja controla pagos y cierre.

Alcance obligatorio y límites. Administrar mesas, carta, productos y recetas; abrir pedidos, agregar consumos y actualizar estados de preparación; dividir la cuenta; registrar medios de pago y cierre de caja. En la versión académica, una receta relaciona un producto de la carta con ingredientes y cantidades para su preparación. No se exige descontar inventario de ingredientes ni gestionar compras.

Recorrido principal obligatorio. Abrir un pedido en una mesa, agregar productos de la carta, pasar sus ítems por preparación y entrega, dividir los consumos en subcuentas, cobrar cada una y cerrar la mesa. Consultar luego el cierre de caja y el detalle del pedido persistido.

Ampliaciones opcionales. Inventario de ingredientes, impresora de cocina, promociones o varias terminales.

3. Investigación

Consulten un restaurante o bar y conversen con personal de atención, cocina y caja. Busquen comprender cómo pasa la información entre esas funciones durante un pedido, sin interferir con el servicio.

Qué observar. Observen apertura de mesa, carga de consumos, envío a preparación, entrega y cobro. Presten atención a pedidos de varias unidades, cuentas separadas y correcciones de un consumo ya enviado o cobrado.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué evento abre un pedido y qué datos relacionan sus consumos con una mesa?
- ¿Cómo describen un producto de la carta y qué cantidades de ingredientes componen su receta?
- ¿Qué estados atraviesa un ítem entre el pedido y la entrega y quién informa cada cambio?
- ¿Cómo dividen una cuenta cuando varias personas pidieron unidades del mismo producto?
- ¿Qué hacen si se cargó un consumo incorrecto antes o después de cobrarlo?
- ¿Cómo registran pagos parciales o por distintos medios sin cobrar dos veces una unidad?
- ¿Cuándo consideran libre una mesa y qué comparan para obtener el cierre de caja?

Materiales que conviene conocer. Revisen carta, receta, comanda, cuenta dividida y cierre por medio de pago. Soliciten modelos sin datos de clientes ni cifras sensibles y distingan el pedido de los movimientos de cobro.

Del hallazgo al requisito. Si la división manual pierde unidades, formulen un requisito que conserve la cantidad pedida y asigne cada unidad una sola vez. Describan Abrir pedido, Actualizar preparación, Dividir y Cobrar, con alternativas por anulación o saldo insuficiente para el importe solicitado.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Mesa tiene pedidos; Pedido contiene ítems con producto, cantidad y precio aplicado. Receta relaciona producto e ingredientes. Cuenta o Subcuenta agrupa unidades de consumo y Pago registra importes y medio utilizado. El equipo decidirá cómo representar divisiones sin copiar consumos como nuevas ventas.

Del análisis al modelo. Identifiquen mesa, producto, ingrediente, cantidad de receta, pedido, ítem, precio aplicado, subcuenta y pago. Expresen pedido–ítem como uno a muchos y receta–ingrediente con las cantidades de la relación. Definan claves primarias y foráneas, campos obligatorios y restricciones positivas. Normalicen carta e ingredientes sin copiar la receta completa en cada consumo.

Maestros, operaciones e historial. Separen carta y recetas editables de consumos, asignaciones de unidades y pagos históricos. ¿Puede darse de baja un producto con pedidos abiertos? ¿Cómo preservan consumos si cambia la carta? ¿Qué se conserva al anular un ítem cobrado? Justifiquen cómo las subcuentas reparten cantidades sin crear ventas nuevas.

Implementación e integración. Creen scripts SQL y datos ficticios con un pedido de tres unidades de un producto repartidas entre subcuentas. Integren CRUD de mesas, productos y recetas y consultas de pedidos/caja. Coordinen división y cobro con transacciones cuando afecten varias tablas; prueben que una unión de detalles y pagos no multiplique el total.

Consultas orientativas y persistencia aplicable. Pedidos pendientes por estado, consumo por mesa, productos vendidos, saldos y cierre de caja por medio de pago.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen el avance de preparación, la división de unidades y el cobro hasta liberar la mesa. Incluyan retroceso inválido de estado, unidad ya asignada, pago excesivo, anulación y error al guardar. Marquen qué confirmación transforma un pedido pendiente en una cuenta saldada.

Clases y responsabilidades. Pedido puede componer ítems y asociarse con Mesa; un servicio coordina subcuentas y pagos. Encapsulen el estado de preparación y las cantidades pendientes de asignar. Distingan el total consumido del total cobrado para no delegar su cálculo a la tabla visual.

Aplicación técnica orientativa. Pedido compone ítems y un servicio coordina división y cobro. STL puede agrupar consumos y calcular subtotales. Qt muestra mesas, comandas y diálogos de cuenta. SQL permite verificar cantidades asignadas y totalizar pagos sin multiplicarlos por uniones incorrectas.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Cantidades positivas y productos habilitados. Utilizar una secuencia clara de estados, por ejemplo pendiente, en preparación, listo y entregado; impedir retrocesos incompatibles. Conservar precios de los consumos. Para dividir, asignar unidades completas de cada ítem a una única subcuenta: la suma asignada no puede superar ni perder la cantidad pedida. Un ítem de tres unidades puede distribuirse, pero cada unidad debe cobrarse una sola vez.
- Registrar pagos sobre el saldo; no aceptar duplicados ni importes superiores a lo adeudado. La mesa se libera cuando todos los consumos válidos están cobrados. La anulación de un consumo conserva el motivo y no permite borrar un cobro ya realizado. El cierre agrupa pagos registrados en una fecha por medio de pago; no vuelve a generar cobros al consultarlo.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con una mesa y un pedido pequeño que pase por preparación, entrega y pago. Después repartan tres unidades entre dos subcuentas y comprueben la conservación del total. Ensayen la última unidad por asignar, un doble clic de cobro y una anulación incompatible.

Evidencias de funcionamiento que deben mostrar. Dividir un pedido con varias unidades; comprobar que subtotales y pagos coinciden con el total; rechazar cobros repetidos; mantener estados y comprobantes al reiniciar.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten mapa o lista de mesas, carta/recetas, pedido y preparación, división de cuenta y cierre de caja. Dibujen el camino desde mesa abierta a consumo, subcuenta y comprobante, incluyendo cómo volver a un pedido pendiente.

Qué representar en los bocetos. Señalen cantidades pedidas, asignadas y restantes, estado de preparación, subtotal y saldo. Prevean mesa sin pedido, carta sin productos habilitados, unidad ya asignada y pago rechazado. Antes de liberar la mesa, muestren qué consumos válidos siguen pendientes de cobro.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. Las señales de preparar, dividir y cobrar invocan servicios diferentes con entradas explícitas. Backend valida transiciones y saldos, datos confirma y Qt refresca. Cambiar el texto de una celda o mover un ítem visualmente no debe modificar por sí solo la cuenta histórica.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-restaurante» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Dividir unidades sin duplicación», terminada cuando la suma asignada coincide con el pedido; «Liberar mesa saldada», terminada cuando solo se libera después de cobrar todos los consumos válidos.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Conserva cantidades al dividir una subcuenta» / «Evita multiplicar pagos en el cierre de caja»

Revisión integrada. Revisen un pedido desde atención, preparación y caja con los mismos datos. Integren primero estados y cobro simple, después división; cada cambio debe conservar precio histórico y totales al reabrir.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan cómo se coordinan atención, cocina y caja, qué confusiones buscan resolver y qué necesitan sus usuarios. Delimiten un local, recetas descriptivas e inventario de ingredientes opcional; expongan beneficios esperados sin afirmar mejoras no medidas. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten casos de pedido, preparación, división y cobro con unidades concretas. Relacionen hallazgos con alternativas de anulación y rechazo por saldo, evitando confundir un caso de negocio con la secuencia de un caso de uso. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen estados, representación de recetas, reparto de unidades, conservación de precios y cálculo del cierre sin duplicar pagos. Expliquen cómo se impide borrar un cobro al anular y cuándo puede liberarse la mesa. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Describan acceso, carga de carta y receta, apertura de mesa, agregado y preparación, reparto de consumos, cobro de subcuentas y cierre. Usen capturas ficticias con unidades repetidas y mensajes por asignación o cobro inválido.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen pedido y preparación antes de dividir la cuenta; registren las decisiones sobre estados, unidades y liberación de mesa junto con pruebas e issues relacionadas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren cantidades, subtotales y pagos de una cuenta dividida, transiciones inválidas, doble confirmación y consulta persistida tras reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.7 Gimnasio

1. Nombre del tema

Gimnasio. Sistema de socios, planes y actividades de gimnasio, con cuotas, cupos, inscripciones, asistencia y consultas de vencimiento.

2. Objetivo

Gestionar socios, planes y asistencia a actividades. Administración mantiene cuotas y oferta; recepción registra inscripciones, pagos y concurrencia.

Alcance obligatorio y límites. Administrar socios, planes, cuotas, actividades y profesores; programar turnos con cupos; inscribir participantes; registrar asistencia y consultar vencimientos. Un turno es una ocurrencia concreta de una actividad con fecha, horario y profesor. La inscripción reserva una plaza para ese turno.

Recorrido principal obligatorio. Registrar un socio, asignarle un plan y registrar su cuota; crear una actividad y un turno; inscribirlo con vigencia válida; controlar el cupo; marcar asistencia; consultar su historial y los próximos vencimientos.

Ampliaciones opcionales. Lista de espera, planes familiares, estadísticas de continuidad o lectura de credenciales.

3. Investigación

Visiten un gimnasio o club que ofrezca actividades con cupos y consulten recepción, administración y profesores. Distingan el control de acceso general del registro de participación en una clase concreta.

Qué observar. Sigan la afiliación a un plan, el registro de cuota, la reserva de plaza y la asistencia. Observen cómo verifican vigencia para una fecha futura y cómo comunican deuda, cupo completo o cancelación.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué datos y condiciones permiten activar un socio y asignarle un plan?
- ¿Qué actividades cubre cada plan y cómo registran el inicio y fin de su vigencia?
- ¿Cómo distinguen el vencimiento del plan del vencimiento o pago de una cuota?
- ¿Cuándo se reserva una plaza y cómo evitan superar el cupo de una clase?
- ¿Qué hacen si un socio o profesor tiene otra actividad en el mismo horario?
- ¿Hasta cuándo puede cancelarse una inscripción y cómo registran la asistencia sin duplicados?
- ¿Qué consultas necesitan para anticipar vencimientos, revisar deudas y conocer asistencia por actividad?

Materiales que conviene conocer. Revisen ficha de socio, condiciones de plan, cuota, agenda de profesores, inscripción y lista de asistencia. Pidan ejemplos vacíos o anonimizados y anoten qué fecha determina cada control.

Del hallazgo al requisito. Transformen la diferencia entre plan vigente y cuota al día en controles separados con mensajes explicables. Escriban Inscribir a turno, Cancelar inscripción y Registrar asistencia, incluyendo cobertura de la fecha de la actividad y conflictos de horario.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Socio contrata un Plan mediante una afiliación con vigencia. Cuota registra período, vencimiento y pago. Actividad tiene turnos y profesores; Inscripción vincula socio y turno. Asistencia corresponde a esa inscripción o participación concreta, no solo al socio.

Del análisis al modelo. Identifiquen socio, plan, afiliación, período de cuota, actividad, profesor, turno e inscripción. En el DER expresen que una actividad tiene ocurrencias con fecha y cada turno reúne inscripciones; asistencia referencia esa participación. Usen claves primarias/foráneas, campos obligatorios y unicidad socio–turno según su estado. Normalicen planes y actividades sin copiar sus descripciones en cada registro de asistencia.

Maestros, operaciones e historial. Distingan oferta de planes y profesores de afiliaciones, cuotas emitidas, inscripciones y asistencias históricas. ¿Qué ocurre si cambia el precio del plan o se desactiva un profesor con turnos futuros? ¿Cómo se conserva una inscripción cancelada? Documenten el efecto de editar vigencias sin alterar cobros ya registrados.

Implementación e integración. Preparen scripts SQL con socios habilitados, planes vencidos, cuotas impagas y un turno casi completo. Integren CRUD y consultas de vencimiento/asistencia. Al confirmar una inscripción, coordinen comprobación de cupo y registro; al cancelar, liberen la plaza sin borrar historial. Prueben referencias inexistentes y asistencia duplicada.

Consultas orientativas y persistencia aplicable. Cupos por actividad, asistencia por socio o período, cuotas pendientes, planes próximos a vencer y turnos de un profesor.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Inscribir comprobando socio activo, actividad admitida, plan, cuota, fecha cubierta, superposiciones y cupo. Representen también cancelación previa al inicio y asistencia única, con caminos de rechazo y error de persistencia.

Clases y responsabilidades. Separar Plan, Afiliación y Cuota ayuda a explicar por qué una persona no puede inscribirse. Un servicio de inscripción coordina esas asociaciones con Turno; la asistencia no debería deducirse de un simple cambio visual de la lista.

Aplicación técnica orientativa. Plan, Afiliación e Inscripción pueden separar condiciones comerciales y participación. Un servicio aplica vigencia y capacidad; std::vector y algoritmos STL permiten ordenar turnos y localizar socios. Qt puede mostrar agenda, ficha del socio y control de asistencia. SQL relaciona pagos, afiliaciones e inscripciones y calcula cupos utilizados.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Definir qué actividades admite cada plan y cómo se expresa su vigencia. En el alcance académico, solo se inscribe un socio activo con plan vigente y cuota del período al día; la cobertura debe incluir la fecha del turno. El cupo cuenta inscripciones activas y se comprueba al confirmar. No duplicar la inscripción del mismo socio ni exceder capacidad.
- Evitar turnos superpuestos del mismo profesor y del mismo socio. La cancelación anterior al inicio libera la plaza sin borrar historial. Registrar asistencia una sola vez por socio y turno. Cambiar el precio de un plan no modifica una cuota ya emitida. El vencimiento del plan y el de la cuota deben distinguirse para explicar correctamente por qué una inscripción se rechaza.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Implementen un socio al día que se inscribe a un turno y registra asistencia. Agreguen rechazos por deuda, vencimiento y cupo. Prueben cobertura que termina antes de la fecha elegida, última plaza y dos turnos contiguos frente a superpuestos.

Evidencias de funcionamiento que deben mostrar. Admitir una inscripción válida; rechazar otra con plan vencido, deuda o turno lleno; cancelar y recuperar un cupo; impedir asistencia duplicada; reiniciar y conservar cuotas, inscripciones e historial.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Propongan ficha de socio con afiliación/cuotas, administración de actividades y profesores, agenda, inscripción y asistencia. Dibujen la navegación desde el socio a un turno elegible y desde el turno a sus participantes y consultas.

Qué representar en los bocetos. Identifiquen vigencia del plan, vencimiento de cuota, estado de pago, cupo disponible y horario. Boceten socio sin plan, clase llena, deuda y profesor ocupado. El motivo de rechazo debe indicar qué condición falla sin presentar todos los casos como «socio vencido».

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de inscripción solicita socio y turno al servicio; este verifica cobertura, deuda y capacidad y persiste. Qt actualiza plazas y participantes al confirmar. La marca de asistencia debe usar la inscripción concreta y recibir confirmación de registro único.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-gimnasio» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Inscripción con vigencia y cuota», terminada al distinguir ambos rechazos; «Asistencia única por turno», terminada cuando repetir la acción no crea una segunda participación.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Verifica cobertura del plan para la fecha del turno» / «Libera cupo al cancelar una inscripción»

Revisión integrada. Alineen datos de afiliación y cuota antes de conectar la agenda. Revisen conjuntamente mensajes, cupos y consulta de asistencia después de reiniciar, utilizando el mismo socio ficticio.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan la necesidad de coordinar recepción, profesores y administración para controlar inscripciones y vencimientos. Limiten el sistema a la oferta académica de planes y turnos y expliquen los beneficios esperados de cupos e historial coherentes. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Relacionen hallazgos con casos de afiliación, cuota, inscripción, cancelación y asistencia. Especifiquen alternativas y fechas relevantes; separen la necesidad del usuario de la regla concreta que debe validar el sistema. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen plan frente a afiliación y cuota, cobertura temporal, actividades admitidas, cupos e incompatibilidades de profesor y socio. Expliquen conservación de importes y restricción de asistencia duplicada. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Muestren acceso, alta del socio, plan y cuota, creación de actividad/turno, inscripción, asistencia e historial. Incluyan mensajes por deuda, plan vencido y turno lleno, además de cancelación con recuperación de plaza.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen afiliaciones y cuotas con agenda y cupos; mantengan visibles las decisiones sobre vigencias y actividades incluidas, junto con tareas y pruebas que dependen de ellas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren un alta válida, cada causa de rechazo, límite de capacidad, cancelación previa al inicio y asistencia repetida, con consulta posterior al reinicio. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.8 Concesionaria

1. Nombre del tema

Concesionaria. Sistema de inventario y operaciones de una concesionaria, con unidades identificables, reservas, documentación y simulación académica de financiación.

2. Objetivo

Organizar el inventario y las operaciones de una concesionaria pequeña. Administración controla unidades y documentación; vendedores o gestores registran clientes, reservas y ventas.

Alcance obligatorio y límites. Administrar vehículos, marcas, modelos, clientes y vendedores o gestores; registrar compras, reservas y ventas; mantener documentación asociada y simular o registrar financiación. Cada vehículo representa una unidad identificable. La documentación se limita a tipo, referencia, estado y observaciones; no se exige validar trámites con organismos externos.

Recorrido principal obligatorio. Registrar una compra y el vehículo; cargar sus datos y documentación; reservarlo para un cliente; simular la financiación; confirmar la venta, registrar las condiciones y consultar el cambio de estado y el historial.

Ampliaciones opcionales. Adjuntos digitales, toma de usados, varios planes financieros o exportación de reportes.

3. Investigación

Consulten una concesionaria pequeña o un local de compraventa y conversen con vendedores, administración y quien revisa documentación. No soliciten legajos reales ni condiciones financieras confidenciales.

Qué observar. Sigan el ingreso de una unidad, su publicación como disponible, la reserva y la confirmación de venta. Observen cómo identifican el vehículo concreto y cómo comunican faltantes documentales o vencimiento de una reserva.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué identifica de forma única una unidad y qué datos registran al incorporarla por compra?
- ¿Cómo vinculan marca y modelo con la unidad y su documentación?
- ¿Qué datos requiere una reserva, cuánto dura y cómo se libera cuando vence o se cancela?
- ¿Qué controles realizan para evitar vender una unidad ya comprometida o vendida?
- ¿Qué documentos o estados consideran indispensables antes de confirmar la operación?
- ¿Qué condiciones de precio, anticipo y cuotas necesitan explicar al cliente y conservar después?
- ¿Qué consultas utilizan para revisar inventario, reservas próximas a vencer y ventas por responsable?

Materiales que conviene conocer. Conozcan una ficha de unidad, un registro de compra, reserva, lista de documentación y resumen de condiciones de venta. Pidan plantillas o ejemplos anonimizados; la fórmula de financiación exigida seguirá siendo la académica de esta ficha.

Del hallazgo al requisito. Transformen la confusión entre modelo y unidad en un requisito de identificación única por vehículo. Describan Incorporar compra, Reservar, Simular financiación y Confirmar venta, con alternativas por documentación pendiente o reserva vencida.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Modelo pertenece a Marca y Vehículo identifica una unidad. Compra incorpora el vehículo; Reserva y Venta lo vinculan con un cliente y responsable. Documento se asocia a la operación o unidad. Una financiación puede contener un plan y sus cuotas, sin reproducir un sistema bancario.

Del análisis al modelo. Identifiquen marca, modelo, unidad, cliente, responsable, compra, reserva, venta y documentación. Expresen marca–modelo–vehículo con sus cardinalidades y relacionen una venta con sus condiciones y cuotas cuando existan. Definan claves primarias/foráneas, identificador de unidad único, campos obligatorios y valores monetarios válidos. Normalicen marcas, modelos y clientes sin repetir sus datos completos en cada documento.

Maestros, operaciones e historial. Separen inventario maestro de compras, reservas y ventas históricas. ¿Qué ocurre al cambiar un vendedor, cancelar una reserva o desactivar un cliente con una compra? ¿Cómo conservan condiciones pactadas si luego cambia el precio? Documenten qué estados de documentación bloquean la confirmación y qué referencia mantiene cada documento.

Implementación e integración. Preparen scripts SQL con unidades disponibles, reservadas, vendidas y con documentación pendiente. Integren CRUD y consultas de inventario/financiación. Coordinen venta, cierre de reserva y estado de unidad en una transacción; si se generan cuotas, su registro debe coincidir con las condiciones confirmadas, sin deuda cuando el capital es cero.

Consultas orientativas y persistencia aplicable. Inventario disponible, reservas por vencer, ventas por responsable, documentación pendiente y detalle de financiación.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Reserva y Confirmación de venta, incluyendo vencimiento, unidad ya vendida, documentación no habilitante y fallo al guardar. Representen la simulación con capital cero, parámetros inválidos y ajuste de redondeo de la última cuota.

Clases y responsabilidades. Vehículo representa la unidad; Modelo describe características compartidas. Un servicio puede coordinar reserva y venta y otro cálculo interno producir la simulación, sin convertirlo en un servicio web. Mantengan estados y condiciones encapsulados; no creen subclases de vehículos solo para separar marcas.

Aplicación técnica orientativa. Vehículo encapsula estado y un servicio coordina reserva y venta. STL puede filtrar inventario y reunir cuotas; herencia solo si aparecen comportamientos realmente distintos. Qt muestra ficha, documentación y simulación. SQL relaciona unidad, cliente, operación y responsable, conservando importes originales.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Identificador de unidad único y precios válidos. Una unidad no puede tener dos reservas activas ni venderse dos veces. La reserva tiene vencimiento documentado; vencida o cancelada, libera el vehículo. Al vender, comprobar disponibilidad y cerrar la reserva correspondiente.
- Para financiación académica, utilizar precio, anticipo, tasa total del plan expresada en porcentaje y cantidad de cuotas: capital es precio menos anticipo; interés es capital multiplicado por tasa y dividido por 100; importe financiado es capital más interés. Las cuotas se distribuyen en partes iguales y el ajuste por redondeo se aplica a la última.
- Anticipo entre cero y precio, tasa no negativa y cantidad positiva; si el capital es cero, no generar deuda. Rotular el cálculo como simulación académica, no como oferta crediticia. Documentar qué estados de documentación impiden confirmar la venta.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con una unidad comprada y una venta sin financiación. Añadan reserva y simulación; comprueben que anticipo más cuotas coincide con precio más interés. Ensayen anticipo igual al precio, una cuota, valores inválidos y reserva exactamente vencida.

Evidencias de funcionamiento que deben mostrar. Rechazar venta de una unidad ya vendida; liberar una reserva vencida; verificar que anticipo y cuotas suman el total pactado; conservar comprador, responsable y documentación al reiniciar.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten inventario filtrable, ficha de unidad/documentación, cliente, reserva y simulación/venta. Tracen la navegación desde una unidad disponible hasta el resumen confirmado y desde allí al historial y las cuotas pactadas.

Qué representar en los bocetos. Muestren identificador, estado, vencimiento de reserva, documentación faltante, precio, anticipo, tasa total y cuotas. Rotulen el cálculo como simulación académica. Prevean inventario vacío, unidad no disponible, capital cero y rechazo de venta por documentación.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de simulación envía parámetros al cálculo C++; el de venta vuelve a validar disponibilidad y documentación antes de persistir. Qt presenta las cuotas devueltas y solo cambia la unidad a vendida después de la confirmación; no toma un resultado visual anterior como autorización de venta.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-concesionaria» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Reserva con vencimiento», terminada cuando una reserva vencida libera la unidad; «Cuotas con ajuste final», terminada cuando suman el importe financiado y capital cero no genera deuda.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Bloquea venta de unidad con documentación pendiente» / «Ajusta redondeo en la última cuota simulada»

Revisión integrada. Revisen inventario, servicio de venta y condiciones guardadas con una misma unidad ficticia. Integren la validación documental antes del cierre de venta y comprueben que editar precios maestros no modifica operaciones anteriores.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Expliquen la necesidad de reconocer disponibilidad real, seguimiento documental y condiciones de venta para administración y vendedores. Justifiquen una concesionaria pequeña y una simulación académica sin validaciones externas ni servicios bancarios. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten compra, reserva, simulación y venta con actores, pasos y alternativas por vencimiento o faltantes. Distingan las prácticas observadas de las condiciones de cálculo

fijadas por la consigna. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.

- **Manual técnico.** Justifiquen identidad de unidad, estados, política documental, fórmula de capital e interés y redondeo final. Expliquen cierre consistente de reserva/venta y conservación de comprador, responsable y condiciones históricas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Muestren acceso, ingreso por compra, ficha y documentos, reserva para un cliente, simulación rotulada, confirmación y consulta histórica. Incluyan reserva vencida, rechazo de doble venta y lectura de cuotas con datos ficticios.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen inventario, documentación y cálculo antes de cerrar ventas; registren los estados documentales elegidos y la representación de importes con sus pruebas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren venta duplicada, documentación pendiente, reserva vencida, anticipo total y cuotas cuyo redondeo requiere ajuste; contrasten los importes antes y después de reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.9 Taller mecánico

1. Nombre del tema

Taller mecánico. Sistema de seguimiento de reparaciones de taller, desde turno y diagnóstico hasta presupuesto autorizado, trabajo realizado y entrega.

2. Objetivo

Seguir reparaciones de un taller desde el turno hasta la entrega. Recepción registra clientes y vehículos; el personal técnico documenta diagnóstico, tareas y repuestos; administración prepara presupuestos y totales.

Alcance obligatorio y límites. Administrar clientes, vehículos, turnos, tareas y repuestos; registrar diagnóstico; crear órdenes de trabajo y presupuestos; cargar mano de obra y materiales utilizados; controlar estados de reparación. Para mantener el alcance viable, los repuestos se administran con descripción y precio; un inventario de compras no es obligatorio.

Recorrido principal obligatorio. Registrar cliente y vehículo, asignar turno, ingresar diagnóstico, elaborar presupuesto y marcar su aceptación. Abrir o actualizar la orden, registrar trabajos y cantidades realmente utilizadas, calcular el total, finalizar la reparación y consultar el historial del vehículo.

Ampliaciones opcionales. Inventario de repuestos, fotografías, avisos al cliente o indicadores de tiempos de reparación.

3. Investigación

Acérquense a un taller mecánico y conversen con recepción, personal técnico y quien elabora presupuestos. Investiguen cómo se comunica la autorización del cliente y cómo se registra lo realmente ejecutado.

Qué observar. Observen recepción del vehículo, diagnóstico, elaboración y aceptación del presupuesto, asignación de tareas y cierre. Presten atención a un repuesto o trabajo adicional que obliga a revisar lo autorizado.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Cómo asignan un turno y qué capacidad de atención consideran antes de aceptarlo?
- ¿Qué datos vinculan vehículo, cliente, motivo de ingreso y diagnóstico?
- ¿Cómo separan el presupuesto estimado de la orden y del trabajo realmente efectuado?
- ¿Cómo registran quién aceptó un presupuesto y qué versión fue autorizada?
- ¿Qué hacen si durante la reparación aparece una tarea o repuesto que excede la autorización?
- ¿Cómo calculan mano de obra y repuestos utilizados sin sumar también la estimación original?
- ¿Qué condiciones permiten finalizar y entregar, y qué información buscan luego en el historial del vehículo?

Materiales que conviene conocer. Revisen agenda, ficha de recepción, diagnóstico, presupuesto con revisión, autorización y orden de trabajo. Soliciten ejemplos vacíos o anonimizados y anoten cómo queda identificada cada versión aceptada.

Del hallazgo al requisito. Conviertan el hallazgo de trabajos adicionales en un caso Revisar presupuesto con nueva aceptación antes de continuar. Describan Ingresar vehículo, Autorizar, Registrar ejecución y Finalizar reparación, con estados, alternativas y resultados verificables.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Cliente tiene vehículos; Turno puede originar una OrdenTrabajo. La orden contiene diagnóstico, tareas y repuestos utilizados. Presupuesto contiene líneas estimadas de repuestos y mano de obra con una versión y estado de aceptación. Deberán distinguir estimación, autorización y ejecución efectiva.

Del análisis al modelo. Identifiquen cliente, vehículo, turno, orden, diagnóstico, tarea, repuesto, presupuesto, versión y aceptación. En el DER expresen cliente–vehículos, orden–líneas realizadas y presupuesto–versiones/líneas según el diseño. Definan claves primarias/foráneas, identificación del vehículo, campos obligatorios, cantidades y precios válidos. Normalicen catálogos de tareas y repuestos sin borrar el valor histórico de las líneas.

Maestros, operaciones e historial. Distingan catálogos editables, estimaciones autorizadas y ejecución efectiva. ¿Qué sucede al modificar un repuesto ya presupuestado? ¿Cómo se corrige una orden cerrada conservando lo anterior? ¿Puede darse de baja un cliente con vehículos e historial? Documenten restricciones y relaciones que preservan versiones aceptadas.

Implementación e integración. Preparen scripts SQL y datos ficticios con un presupuesto original y uno revisado, una orden en reparación y otra cerrada. Integren CRUD y consultas de historial. Coordinen aceptación de versión, estado de orden y líneas relacionadas mediante transacciones cuando corresponda; prueben que un error no deja trabajos autorizados solo parcialmente.

Consultas orientativas y persistencia aplicable. Órdenes por estado, agenda, historial de un vehículo, repuestos utilizados y diferencias entre presupuesto aceptado y total final.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen desde diagnóstico hasta autorización y ejecución, con decisión de trabajo adicional y retorno a revisión presupuestaria. Representen Finalizar reparación con cálculo sobre lo realmente realizado, estado incompatible y falla de persistencia.

Clases y responsabilidades. OrdenTrabajo puede componer tareas y consumos; Presupuesto expresa una estimación versionada y su aceptación. Un servicio coordina transiciones, sin que el botón de estado habilite cualquier salto. Diferencien asociación al vehículo de composición de líneas de la orden.

Aplicación técnica orientativa. OrdenTrabajo compone tareas y consumos; un servicio calcula importes y valida estados. Un std::vector permite reunir líneas y algoritmos STL buscar u ordenar reparaciones. Qt puede separar recepción, presupuesto y seguimiento. SQL relaciona vehículo, orden y líneas y permite comparar estimación con ejecución.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Turnos con inicio anterior al fin, sin superar la capacidad de atención que el equipo haya definido. El vehículo debe estar identificado y asociado a un cliente. Usar estados claros, por ejemplo recibida, diagnosticada, presupuestada, autorizada, en reparación, finalizada y entregada; impedir transiciones incompatibles. No iniciar tareas presupuestadas sin aceptación.
- Si el trabajo excede lo autorizado, registrar una revisión del presupuesto y nueva aceptación antes de continuarlo. El total se calcula con cantidades y precios aplicados a repuestos más mano de obra realizada, expresada como horas por tarifa o importe por tarea según la decisión documentada. No sumar simultáneamente presupuesto y trabajo real.
- Conservar versiones aceptadas y valores históricos; una orden cerrada no se modifica sin un mecanismo justificado de corrección.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con una reparación de una tarea y un repuesto autorizados, calculen el total real y consulten historial. Agreguen una revisión por trabajo adicional. Prueben cantidades cero, falta de aceptación, capacidad de turnos agotada y modificación de una orden cerrada.

Evidencias de funcionamiento que deben mostrar. Completar una reparación; rechazar un cambio de estado inválido; modificar el trabajo mediante presupuesto revisado; verificar el cálculo final sin duplicaciones; reiniciar y recuperar documentos y estado.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten agenda/recepción, ficha del vehículo, diagnóstico, presupuesto versionado y seguimiento de orden. Dibujen el paso de presupuesto pendiente a aceptado, luego a ejecución y consulta histórica, mostrando claramente qué ventana registra cada hecho.

Qué representar en los bocetos. Identifiquen versión, estado de aceptación, cantidades estimadas/realizadas, mano de obra y total. Prevean vehículo sin historial, turno sin capacidad, presupuesto rechazado y trabajo adicional pendiente de autorización. Soliciten confirmación al cerrar y expliquen qué ya no se podrá editar libremente.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de finalizar invoca el servicio con la orden, no con un total editable por el usuario. Backend valida autorización y estado, calcula sobre líneas reales y persiste; Qt muestra el resultado y las diferencias respecto del presupuesto sin sumar ambos importes.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-taller» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Revisión con nueva aceptación», terminada cuando un adicional no puede ejecutarse antes de autorizarlo; «Total de ejecución», terminada cuando incluye materiales y mano de obra real sin sumar el presupuesto.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Conserva versiones aceptadas del presupuesto» / «Valida transición de orden a reparación finalizada»

Revisión integrada. Integren recepción, autorización y ejecución con una orden ficticia compartida. Revisen conjuntamente estados, versiones e importes y mantengan el historial del vehículo disponible tras reiniciar.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan la necesidad de conocer estado, autorización y costo real de una reparación para recepción, técnicos y administración. Delimiten capacidad de atención y catálogo de repuestos; el inventario de compras sigue siendo opcional. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten el circuito de autorización y revisión con actor, aceptación, tareas permitidas y alternativas. Vinculen los problemas investigados con requisitos verificables de seguimiento y preservación de versiones. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.

- **Manual técnico.** Justifiquen estados, capacidad de turnos, versiones aceptadas, política de corrección y cálculo de mano de obra. Expliquen por qué el presupuesto no se suma a la ejecución y cómo se preservan precios y relaciones históricas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Muestren acceso, cliente/vehículo, turno, diagnóstico, presupuesto y aceptación; continúen con tareas y materiales reales, revisión autorizada, finalización e historial. Incluyan capturas ficticias y mensajes por falta de autorización o cambio inválido de estado.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen presupuesto y estados antes de ejecutar tareas; registren la elección de horas por tarifa o importe por tarea, y relacionen cambios de decisión con diagramas, código y pruebas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren una reparación completa, un salto de estado rechazado, trabajo adicional autorizado por nueva versión y total sin duplicación, incluyendo recuperación al reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.10 Hotel

1. Nombre del tema

Hotel. Sistema de reservas y estadías de hotel, con disponibilidad por noches, consumos, pagos, ingreso, egreso y limpieza.

2. Objetivo

Gestionar alojamiento y cobros de un hotel pequeño. Recepción administra reservas, ingreso y salida; administración mantiene habitaciones y tarifas y consulta ocupación.

Alcance obligatorio y límites. Administrar habitaciones, tipos, tarifas y huéspedes; consultar disponibilidad y registrar reservas; realizar check-in (ingreso) y check-out (egreso); cargar consumos, pagos y estado de limpieza. El alcance utiliza una habitación por reserva y fechas de alojamiento por noche, sin reservas por horas ni servicios externos.

Recorrido principal obligatorio. Buscar disponibilidad, reservar para un huésped, registrar el check-in, añadir un consumo, cobrar el saldo y efectuar el check-out. La habitación queda pendiente de limpieza; luego se marca limpia y puede admitir un nuevo ingreso compatible.

Ampliaciones opcionales. Varias habitaciones por reserva, tarifas por temporada, reservas en línea o servicios adicionales complejos.

3. Investigación

Consulten un hotel pequeño, hostería o alojamiento y conversen con recepción, administración y personal que informa limpieza. Investiguen qué información comparten sin solicitar datos identificatorios de huéspedes reales.

Qué observar. Sigan la búsqueda por fechas, confirmación de reserva, ingreso y egreso. Observen cómo se comunica que una habitación está sucia aunque no tenga ocupación activa y cómo se consulta una reserva futura.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué fechas, cantidad de huéspedes y tipo de habitación necesitan para buscar disponibilidad?
- ¿Cómo cuentan las noches y cómo tratan una salida y una nueva entrada en la misma fecha?
- ¿Qué datos y tarifa se conservan al confirmar la reserva, aunque después cambien los precios?
- ¿Qué revisan al ingresar y qué hacen si la habitación está desocupada pero pendiente de limpieza?
- ¿Cómo agregan consumos y pagos y cómo identifican el saldo pendiente de la estadía?
- ¿Qué pasos completan para efectuar el egreso y habilitar después la habitación para otro huésped?
- ¿Qué información necesitan consultar sobre llegadas, salidas, cancelaciones y ocupación futura?

Materiales que conviene conocer. Revisen calendario de reservas, ficha de ingreso, cuenta de habitación, comprobante de pago y planilla de limpieza. Soliciten modelos vacíos o anonimizados y distingan fechas reservadas, ocupación efectiva y condición operativa.

Del hallazgo al requisito. Transformen la diferencia entre disponibilidad futura y limpieza actual en requisitos separados. Elaboren Reservar, Realizar check-in, Cargar consumo y Realizar check-out con alternativas por superposición, capacidad, habitación sucia o saldo pendiente.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si

no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Habitación pertenece a un Tipo. Reserva vincula habitación, huéspedes e intervalo; Estadía registra ocupación efectiva. Consumo y Pago se asocian a la estadía o cuenta. La disponibilidad por fechas y la condición operativa de la habitación son conceptos diferentes.

Del análisis al modelo. Identifiquen habitación, tipo, capacidad, tarifa pactada, huésped, reserva, fechas, estadía, consumo y pago. En el DER distingan reserva de ocupación efectiva y relacionen cuenta con consumos/pagos. Definan claves primarias/foráneas, campos obligatorios y restricciones de fechas y capacidad. Normalicen tipos y datos maestros, conservando la tarifa nocturna de la operación.

Maestros, operaciones e historial. Separen habitaciones y tarifas de reservas, estadías y movimientos históricos. ¿Cómo cancelar sin borrar los datos de la reserva? ¿Qué ocurre al deshabilitar una habitación con futuras llegadas? ¿Cómo se conserva una cuenta al modificar datos del huésped? Documenten qué cambios son admisibles después del egreso.

Implementación e integración. Preparen scripts SQL y datos ficticios con reservas contiguas, una superposición, habitación sucia y cuenta con saldo. Integren CRUD y consultas de disponibilidad. Coordinen validación/alta de reserva y los cambios de check-in o check-out en transacciones cuando corresponda, sin dejar ocupación o pagos parcialmente confirmados.

Consultas orientativas y persistencia aplicable. Disponibilidad por período, llegadas y salidas, ocupación, habitaciones pendientes de limpieza e ingresos registrados.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Reservar con entrada incluida/salida excluida; después check-in con reserva vigente, capacidad, habilitación, limpieza y ausencia de ocupación. En el egreso incluyan saldo, pagos únicos y paso a pendiente de limpieza, con errores de persistencia.

Clases y responsabilidades. Reserva, Estadía y Cuenta expresan responsabilidades distintas; asócienlas con Habitación y Huésped y compongan los consumos según su modelo. Un servicio coordina ingreso/egreso sin equiparar el estado de limpieza con toda la disponibilidad de fechas.

Aplicación técnica orientativa. Reserva, Estadía y Cuenta separan disponibilidad, ocupación y cobro. STL puede ordenar reservas o reunir consumos. Qt puede presentar calendario, ficha y cuenta del huésped. SQL busca conflictos de fechas y consolida noches, consumos y pagos.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Usar intervalos de fechas con entrada incluida y salida excluida: una reserva del 10 al 12 ocupa las noches del 10 y 11. La salida debe ser posterior a la entrada. Reservas activas de una misma habitación no pueden superponerse; cancelar libera fechas y conserva historial. Al confirmar, comprobar disponibilidad en la misma operación que guarda la reserva. Respetar capacidad de huéspedes.
- La tarifa pactada por noche se conserva y el total combina noches y consumos válidos; los pagos disminuyen el saldo sin duplicarse. El check-in exige reserva vigente para la fecha de ingreso, habitación sin ocupación activa, habilitada y limpia. El check-out exige cuenta saldada en este modelo y deja la habitación sucia o pendiente de limpieza.
- Una reserva futura puede existir mientras la habitación está ocupada hoy, si los intervalos son compatibles; la limpieza se verifica al ingresar.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Implementen primero una reserva de dos noches y su consulta, luego ingreso, un consumo, pago y egreso. Añadan limpieza y la siguiente reserva contigua. Prueben salida igual a entrada, capacidad excedida, check-in de habitación sucia y check-out con saldo.

Evidencias de funcionamiento que deben mostrar. Admitir reservas contiguas y rechazar superpuestas; bloquear un check-in por limpieza; calcular noches correctamente; cerrar una estadía sin cobros repetidos; recuperar reservas e historial entre ejecuciones.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten disponibilidad por período, reserva/huésped, recepción de ingresos y salidas, cuenta y limpieza. Dibujen cómo una reserva abre la estadía y cómo desde la cuenta se llega al egreso y luego al listado de habitaciones pendientes.

Qué representar en los bocetos. Muestren fechas, noches, capacidad, tarifa pactada, consumos, pagos y saldo. Boceten búsqueda sin disponibilidad, reserva futura sobre habitación hoy ocupada, habitación pendiente de limpieza y egreso bloqueado. Diferencien etiquetas de reserva, ocupación y limpieza.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. Los slots de ingreso y egreso invocan servicios que revisan condiciones actuales y confirman cambios relacionados. Qt presenta la cuenta calculada y el estado devuelto; no libera la habitación simplemente al cerrar el diálogo ni vuelve a cobrar al consultar el resumen.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-hotel» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Reservas contiguas por noche», terminada al aceptar salida/entrada en la misma fecha y rechazar solapamiento; «Egreso con saldo cero», terminada cuando la cuenta se salda y la habitación queda pendiente de limpieza.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Separa limpieza de disponibilidad por fechas» / «Conserva tarifa nocturna de la reserva»

Revisión integrada. Integren reserva, recepción y cuenta con la misma estadía ficticia. Revisen juntos el cómputo de noches, los saldos y la habitación después de egresar y reiniciar, antes de ampliar reportes.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Expliquen la coordinación entre recepción, administración y limpieza y los errores que busca evitar la solución. Delimiten una habitación por reserva y fechas por noche; justifiquen la viabilidad de un hotel pequeño sin servicios externos. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten casos de reserva, cancelación, ingreso, consumos, pago, egreso y limpieza con actores y alternativas. Vinculen observaciones con condiciones comprobables y distingan los límites académicos de posibles prácticas reales más complejas. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen intervalo nocturno, separación reserva/estadía, preservación de tarifa, cálculo de saldo y condición de limpieza al ingresar. Expliquen transacciones y políticas de cambios sobre reservas y

cuentas históricas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Muestren acceso, alta de habitación/tarifa/huésped, búsqueda, reserva, ingreso, consumo, pago y egreso. Continúen con limpieza y nueva entrada compatible; incluyan capturas ficticias, saldo pendiente y recuperación de historial.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen calendario, cuenta y estados de recepción; registren las decisiones sobre fechas, capacidad y cambios posteriores al egreso para mantener alineados modelo, manual y pruebas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren dos noches correctamente contadas, reservas contiguas y superpuestas, bloqueo por limpieza, egreso con deuda y cierre sin pagos repetidos, antes y después de reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.11 Torneo de tatetí

1. Nombre del tema

Torneo de tatetí. Aplicación de tatetí jugable que organiza un torneo local, conserva jugadas y resultados y calcula posiciones y clasificación.

2. Objetivo

Organizar una competencia local en la que las partidas se jueguen dentro de la aplicación. El administrador registra jugadores y torneo; dos participantes alternan jugadas en la misma computadora.

Alcance obligatorio y límites. Programar tatetí; administrar jugadores y torneos; generar partidas y fases; registrar resultados; mostrar tabla de posiciones y clasificación tipo mundial. La modalidad académica será de ocho jugadores en dos grupos de cuatro, todos contra todos dentro de cada grupo; los dos primeros avanzan a semifinales cruzadas y luego a la final.

Recorrido principal obligatorio. Inscribir jugadores, formar grupos, generar encuentros, jugar una partida desde tablero vacío hasta resultado, actualizar posiciones y clasificar. Para la demostración puede utilizarse un torneo parcialmente disputado con datos de prueba, pero se deberá completar en vivo una partida y mostrar el avance que produce.

Ampliaciones opcionales. Más participantes, formatos configurables, reanudación, juego en red o rival automático.

3. Investigación

Consulten jugadores, docentes o responsables de actividades recreativas y torneos escolares. Observen partidas y comparen juegos existentes para reconocer turnos, resultados y claridad del cuadro de competencia; registren qué comparación realizaron.

Qué observar. Sigan una partida completa, incluida una jugada inválida, y cómo un organizador actualiza posiciones. Observen cómo se publican emparejamientos y qué información necesita quien debe jugar el siguiente encuentro.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Cómo identifican e inscriben participantes y cómo comunican grupo y encuentro asignados?
- ¿Cómo se indica quién comienza y cómo reconoce cada persona que es su turno?
- ¿Qué respuesta esperan cuando se elige una casilla ocupada o se intenta jugar después del final?
- ¿Qué datos necesitan conservar para explicar cómo terminó una partida?
- ¿Cómo consultan puntos y desempates y qué decisión necesita quedar visible para los jugadores?
- ¿Qué hacen ante un empate en eliminación y cómo distinguen el resultado jugado del criterio de avance?
- ¿Cómo continúan organizando el torneo si la aplicación se cierra antes de terminar una partida?

Materiales que conviene conocer. Revisen formularios de inscripción, cuadros de grupos, actas de partido, tablas de posiciones y pantallas de tablero. Usen nombres ficticios; una observación de juego no equivale a una entrevista que no se realizó.

Del hallazgo al requisito. Conviertan dificultades para explicar un desempate en requisitos de resultados y decisiones consultables. Elaboren Inscribir, Generar encuentros, Registrar jugada y Clasificar. Comparar reglamentos no modifica la modalidad académica de ocho jugadores ni sus reglas de avance.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si

no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Torneo contiene fases y grupos; Inscripción relaciona jugadores con el torneo; Partida vincula dos participantes y contiene jugadas con posición, símbolo y orden. El tablero representa el estado jugable; la tabla de posiciones se obtiene de resultados válidos.

Del análisis al modelo. Identifiquen torneo, jugador, inscripción, grupo, fase, encuentro, partida y jugada con orden, símbolo y posición. Representen cardinalidades entre participantes y torneo, dos participantes por partida y múltiples jugadas por partida. Traduzcan con claves primarias/foráneas y restricciones de inscripción, orden y coordenadas válidas. Eviten repetir datos personales del jugador en cada jugada; justifiquen normalización y representación del tablero.

Maestros, operaciones e historial. Separen jugadores y torneos aún configurables de encuentros, jugadas y resultados históricos. ¿Qué pasa si se desactiva un jugador inscrito? ¿Cuándo se puede editar un grupo? ¿Cómo se conserva una repetición empatada y la decisión de quién avanza? No permitan que editar una ficha cambie victorias ya registradas.

Implementación e integración. Preparen scripts SQL con ocho participantes ficticios, grupos y encuentros de prueba. Integren CRUD de jugadores/torneos no iniciados y consultas de posiciones. Guarden cada jugada aceptada; coordinen cierre, resultado y avance cuando corresponda en una transacción, diferenciando partida interrumpida de finalizada y evitando contabilizar dos veces un cierre.

Consultas orientativas y persistencia aplicable. Posiciones por grupo, clasificación, historial por jugador y detalle de una partida. Guardar participantes y configuración al crearla, cada jugada aceptada y el cierre con su resultado en una operación consistente.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Registrar jugada: comprobar partida activa, turno, coordenada y casilla, aplicar símbolo, buscar victoria/empate y cambiar turno o cerrar. Dibujen además clasificación de grupos, semifinales cruzadas y empate de eliminación con una sola repetición y criterio de prioridad.

Clases y responsabilidades. Partida protege tablero, jugador actual y resultado; Torneo coordina grupos y fases. Diferencien una partida jugada de un encuentro que puede necesitar repetición. El orden de posiciones debe responder a una comparación documentada y no al orden accidental de filas en pantalla.

Aplicación técnica orientativa. Partida controla turnos y resultado; Torneo coordina fases. std::array puede representar el tablero y std::sort ordenar posiciones. Qt conecta botones con jugadas. SQL relaciona encuentros, jugadores y movimientos y calcula resultados sin duplicar partidas.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Tablero de 3 por 3; alternancia de X y O; jugada solo en casilla libre; gana quien alinea tres símbolos y hay empate al completar el tablero sin ganador. No aceptar jugadas posteriores al cierre. En grupos: victoria 3 puntos, empate 1 y derrota 0. Desempatar por victorias, resultado directo si quedan exactamente dos igualados y, finalmente, orden de inscripción.
- Semifinales: primero de un grupo contra segundo del otro. En eliminación, un empate provoca una única repetición; si vuelve a empatarse, avanza quien tenga mejor prioridad según puntos de grupos, victorias y orden de inscripción. Registrar esta decisión sin convertir el empate en una victoria jugada. Definir el jugador inicial por el orden publicado del encuentro; invertirlo en la repetición.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Programen primero una partida desde tablero vacío, con victoria y empate comprobables sin interfaz final. Persistan jugadas y cierre, conecten el tablero y luego incorporen grupos y clasificación. Prueben última casilla, intento posterior al cierre y dos empates sucesivos en eliminación.

Evidencias de funcionamiento que deben mostrar. Rechazar casilla ocupada y turno inválido; detectar victoria y empate; consultar jugadas persistidas; actualizar posiciones una sola vez y comprobar desempate y avance tras reiniciar.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten administración de jugadores, configuración de torneo, grupos/encuentros, tablero, resultado y clasificación. Dibujen cómo un encuentro pendiente abre la partida y cómo el cierre vuelve al cuadro actualizado y permite consultar el detalle histórico.

Qué representar en los bocetos. Muestren participantes, símbolos, turno, fase y resultado, con casillas y acciones reconocibles. Prevean torneo incompleto, encuentro no habilitado, casilla ocupada, empate y partida interrumpida. Expongan el motivo del avance por prioridad sin presentar el empate como victoria jugada.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. Cada botón de casilla envía su posición al servicio de partida. Backend valida, determina el resultado y persiste; Qt representa el tablero confirmado. Abrir de nuevo las posiciones solo consulta resultados, sin sumar puntos ni avanzar otra vez al ganador.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-torneo-tateti» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Cierre único de partida», terminada cuando reabrir el resultado no agrega puntos; «Desempate de eliminación», terminada cuando una repetición empatada registra prioridad y conserva ambos empates.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Rechaza jugadas después del cierre del tatetí» / «Aplica semifinales cruzadas desde posiciones de grupos»

Revisión integrada. Integren tablero, jugadas y resultado antes del cuadro del torneo. Revisen con otro integrante una secuencia conocida, la tabla calculada y el avance tras reiniciar; mantengan datos precargados diferenciados de la partida que se juega en vivo.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Presenten la finalidad recreativa u organizativa: ofrecer partidas jugables y un torneo comprensible para participantes y organizadores. Describan la necesidad de reglas visibles y resultados recuperables; justifiquen el formato local de ocho personas sin forzar un objetivo comercial. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten observaciones y casos de inscripción, partida y clasificación, con actores y caminos por jugada inválida, empate e interrupción. Vinculen cada regla de puntuación y avance con una prueba identificable. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.

- **Manual técnico.** Justifiquen tablero, validación del turno, detección de final, desempates, repetición única e inversión del inicio. Expliquen almacenamiento de jugadas, cierre consistente y cálculo de posiciones sin duplicación, incluyendo qué ocurre con partidas interrumpidas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Muestren login, jugadores, torneo, grupos y encuentros; jueguen una partida completa, consulten su detalle, posiciones y clasificación. Incluyan capturas ficticias, casilla inválida, empate y una explicación de la prioridad de avance cuando corresponda.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Planifiquen partida, persistencia y competencia como avances integrados; registren decisiones de representación de encuentro/repetición y vinculen cada criterio de desempate con tareas y evidencias. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren secuencias válidas, casilla ocupada, turno inválido, victoria, empate y clasificación con igualdad de puntos, verificando jugadas y posiciones después del reinicio. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.12 Gestión clínica

1. Nombre del tema

Gestión clínica. Sistema académico de agenda e historia de atención de una clínica médica o veterinaria, con prácticas y pagos sobre datos exclusivamente ficticios.

2. Objetivo

Organizar turnos e historia de atención de una clínica médica o veterinaria. El equipo deberá elegir una modalidad y sostenerla en todo el sistema. Recepción administra agenda y pagos; profesionales registran atenciones. Se utilizarán exclusivamente datos ficticios.

Alcance obligatorio y límites. Administrar pacientes, profesionales, especialidades, turnos y prácticas; registrar historia de atención y pagos. En veterinaria, administrar animales y responsables y vincularlos explícitamente. La aplicación registra información académica; no deberá incorporar diagnóstico automático, indicación terapéutica ni integración con prestadores externos.

Recorrido principal obligatorio. Registrar un paciente y, cuando corresponda, su responsable; asignar un turno con un profesional; abrir la atención; registrar observaciones y prácticas; calcular y registrar el pago; consultar la historia completa desde la ficha del paciente.

Ampliaciones opcionales. Adjuntos ficticios, recordatorios, varios responsables por animal o estadísticas ampliadas de atención.

3. Investigación

Elijan modalidad médica o veterinaria y consulten recepción, administración y profesionales de ese contexto. Investiguen procedimientos y modelos vacíos, sin acceder a historias reales, diagnósticos personales ni información confidencial.

Qué observar. Observen cómo se solicita y confirma un turno, se identifica al paciente y se registra una atención y su pago. En veterinaria distingan siempre al animal atendido de la persona responsable y cómo se selecciona al paciente correcto.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Qué datos necesitan para identificar al paciente y, en veterinaria, a su responsable?
- ¿Cómo asignan profesional y especialidad y verifican horarios disponibles?
- ¿Qué hacen cuando el profesional o el paciente ya tiene un turno superpuesto?
- ¿Cómo distinguen un turno cancelado, pendiente y una atención efectivamente realizada?
- ¿Qué fecha, profesional, observaciones y prácticas necesitan recuperar de cada visita?
- ¿Cómo corrigen una anotación anterior conservando su referencia y sin sobrescribir toda la historia?
- ¿Cómo registran importes, pagos y saldos y qué consultas necesitan recepción y profesionales?

Materiales que conviene conocer. Soliciten una agenda vacía, ficha de paciente, formato de atención, catálogo de prácticas y recibo interno de ejemplo. En veterinaria agreguen ficha del animal y responsable; no fotografíen ni copien historias clínicas reales.

Del hallazgo al requisito. Transformen la necesidad de consultar visitas anteriores en registros de atención separados y vinculados. Escriban Asignar turno, Registrar atención, Corregir registro y Registrar pago con actores y alternativas, sin incorporar decisiones diagnósticas ni terapéuticas automáticas.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si

no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Profesional se vincula con Especialidad y Turno relaciona profesional y paciente. Atención contiene fecha, observaciones y prácticas realizadas. Pago se asocia con la atención o cuenta correspondiente. En veterinaria, Paciente es el animal; Responsable es la persona de contacto. Se podrá delimitar un responsable principal por animal, con varios animales a su cargo.

Del análisis al modelo. Identifiquen paciente, profesional, especialidad, turno, atención, práctica e importe aplicado. Expresen paciente–atenciones como uno a muchos y atención–prácticas mediante la relación adecuada. En veterinaria modelen animal–responsable explícitamente. Definan claves primarias/foráneas, fechas obligatorias y valores válidos; normalicen profesionales y prácticas sin duplicar su descripción en toda la historia.

Maestros, operaciones e historial. Distingan catálogos y fichas administrables de atenciones, correcciones y pagos históricos. ¿Qué ocurre al desactivar un paciente o profesional con atenciones? ¿Cómo se referencia una corrección sin borrar el registro anterior? ¿Cómo afecta cancelar un turno a la historia? Conserven precios aplicados y eviten confundir cancelación con atención realizada.

Implementación e integración. Creen scripts SQL con pacientes ficticios que tengan varias visitas, turnos cancelados y pagos pendientes. Integren CRUD, agenda e historia. Coordinen atención y prácticas, y pago/saldo cuando corresponda, mediante transacciones; prueben referencias inexistentes, doble confirmación y preservación de la historia ante bajas.

Consultas orientativas y persistencia aplicable. Agenda por profesional o especialidad, historia por paciente, prácticas por período, pagos y saldos. En veterinaria, listado de pacientes de un responsable.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Representen Asignar turno con controles de profesional/paciente activos y superposición de ambos. Dibujen atención–prácticas–pago con validación de importes, corrección trazable y falla de escritura. El diagrama debe mostrar por qué cancelar no crea una atención.

Clases y responsabilidades. Paciente, Turno y Atención tienen responsabilidades distintas; un servicio coordina agenda y otro el registro de atención según el diseño elegido. La historia se consulta como conjunto de hechos, no como un atributo reemplazado en cada visita. Usen asociaciones explícitas con profesional y prácticas.

Aplicación técnica orientativa. Paciente, Turno y Atención expresan responsabilidades distintas; un servicio valida la agenda. STL permite ordenar atenciones o buscar turnos. Qt puede ofrecer ficha, agenda y diálogo de

atención. SQL relaciona historia, profesional y prácticas; en veterinaria permite consultar animales por responsable sin confundir ambas identidades.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Paciente y profesional activos; horario válido y compatible con la agenda. Impedir superposición de turnos activos del mismo profesional y del mismo paciente. Un turno cancelado no debe aparecer como atención realizada. Registrar fecha y profesional de cada atención y conservar las prácticas y sus importes aplicados aunque cambien precios posteriores.
- La historia es un conjunto de registros vinculados, no un único texto que se sobrescribe en cada visita. Las correcciones deberán conservar referencia al registro anterior o un mecanismo documentado equivalente. El pago no puede superar el importe pendiente ni registrarse dos veces por la misma confirmación. No dar de baja físicamente pacientes o profesionales si eso elimina relaciones históricas necesarias.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con un paciente, un turno y una atención con una práctica; registren su pago y recuperen la historia. Agreguen una segunda visita, cancelación y corrección. Prueben turnos contiguos/superpuestos, importe superior al saldo y profesional inactivo.

Evidencias de funcionamiento que deben mostrar. Completar turno, atención y pago; rechazar una superposición; conservar varias atenciones de un mismo paciente; recuperar historia al reiniciar; verificar que la modalidad elegida aparece coherentemente en pantallas, datos y documentación.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten ficha de paciente, agenda por profesional, registro de atención, cuenta y consulta de historia. En veterinaria, dibujen la selección de animal desde su responsable. El mapa debe permitir abrir una visita concreta y regresar al conjunto histórico.

Qué representar en los bocetos. Muestren paciente, profesional, fecha, estado del turno, prácticas, importes y saldo. Prevean historia sin visitas, turno incompatible, registro corregido y pago rechazado. Mantengan nombres y etiquetas coherentes con la modalidad elegida y datos ficticios en todas las capturas.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de guardar atención envía observaciones y prácticas al servicio, que valida relaciones y persiste un nuevo registro. Qt refresca la historia completa; no sobrescribe visitas anteriores. Cancelar turno y registrar pago deben invocar sus propios casos de uso con resultados explícitos.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-gestion-clinica» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Historia por visitas», terminada cuando dos atenciones se consultan sin sobrescribirse; «Agenda sin superposición», terminada cuando controla tanto profesional como paciente y conserva turnos cancelados.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Vincula corrección con la atención anterior» / «Rechaza pago mayor al saldo de la atención»

Revisión integrada. Fijen la modalidad antes de integrar fichas y modelo. Revisen una atención desde agenda hasta historia y cuenta, comprobando que correcciones, cancelaciones y bajas no destruyan relaciones previas.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan la necesidad de organizar agenda e información de atención para recepción y profesionales, dentro de una simulación académica. Justifiquen la modalidad elegida y sus límites, sin presentar el programa como sistema de decisión clínica ni usar información real sensible. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten el relevamiento de procedimientos, los casos de turno, atención, corrección y pago y sus alternativas. En veterinaria, expliquen actores y relación entre responsable y animal;

no atribuyan hallazgos a personas no consultadas. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.

- **Manual técnico.** Justifiquen modalidad, integridad paciente–profesional–atención, controles de agenda, historia acumulativa y correcciones trazables. Expliquen conservación de importes, bajas lógicas y confirmación única de pagos. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Muestren acceso, alta de paciente/responsable cuando corresponda, turno, apertura de atención, prácticas, pago e historia. Incluyan segunda visita, corrección y mensajes de conflicto, con ejemplos totalmente ficticios.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen modalidad, agenda e historia antes de implementar formularios; registren decisiones sobre correcciones y bajas y su relación con requisitos, pruebas y documentación. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren superposición de profesional y paciente, turno cancelado sin atención, varias visitas conservadas, corrección vinculada y pago duplicado o excesivo, verificando recuperación al reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.13 Batalla naval

1. Nombre del tema

Batalla naval. Juego local de batalla naval para dos participantes, con preparación privada de flotas, historial de disparos, ranking y torneo de cuatro jugadores.

2. Objetivo

Ofrecer un juego local por turnos con historial y organización de competencias. El administrador gestiona usuarios y torneos; dos jugadores comparten la computadora con una pantalla de transición entre turnos.

Alcance obligatorio y límites. Programar ubicación de flota, disparos y resultados; gestionar tableros, flotas, turnos, usuarios, historial, ranking y torneos. La modalidad académica utiliza tableros de 6 por 6 y una flota por jugador con barcos de longitudes 3, 2 y 2. Un torneo local de cuatro jugadores se resuelve con dos semifinales y una final, mediante eliminación directa.

Recorrido principal obligatorio. Registrar participantes, crear partida, ubicar ambas flotas en privado, alternar turnos y disparar hasta hundir la flota contraria. Guardar el resultado, actualizar historial y ranking, y avanzar al ganador dentro del torneo. Se pueden preparar semifinales terminadas para demostrar el cierre de una fase.

Ampliaciones opcionales. Tamaños configurables, reanudación, rival automático o juego en red.

3. Investigación

Consulten jugadores, docentes, clubes o espacios recreativos que utilicen juegos de estrategia por turnos. Comparen partidas existentes para analizar preparación, ocultamiento y devolución de disparos, sin adoptar automáticamente reglas distintas a las de esta ficha.

Qué observar. Observen la ubicación de barcos y el cambio de participante en una computadora compartida. Revisen qué información debe conocer cada persona para decidir y qué información revelaría indebidamente la posición de la flota rival.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Cómo explican al comenzar el tamaño del tablero, la flota y las posiciones válidas?
- ¿Qué ayuda necesita un jugador para ubicar un barco sin salir del tablero ni superponerlo?
- ¿Cómo confirmarían la flota antes de comenzar y cómo impedirían ver la preparación del rival?
- ¿Qué debe mostrarse después de agua, impacto o hundimiento y cuándo cambia el turno?
- ¿Qué respuesta esperan al disparar otra vez a la misma coordenada?
- ¿Cómo comprueban quién ganó y qué datos permitirían explicar luego el resultado?
- ¿Cómo organizan semifinales, final y ranking y cómo distinguen una partida interrumpida de una terminada?

Materiales que conviene conocer. Examinen reglamentos, tableros, hojas de seguimiento de disparos, cuadro de torneo y pantallas de transición. Registren comparaciones como observaciones y usen participantes ficticios en las pruebas.

Del hallazgo al requisito. Transformen el riesgo de mostrar posiciones rivales en un requisito de vistas separadas y transición oculta. Escriban Ubicar flota, Confirmar preparación, Disparar y Cerrar encuentro con alternativas por ubicación inválida, repetición y partida finalizada.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si

no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Usuario o Jugador participa en Partida; cada participante tiene Tablero y Barcos con posiciones. Disparo registra atacante, coordenada, turno y resultado. Torneo relaciona encuentros y ganadores. La información completa de la flota no debe confundirse con la vista permitida al adversario.

Del análisis al modelo. Identifiquen jugador, partida, participante, tablero, barco, casillas, disparo, turno y torneo. Expresen dos participantes por partida, varios barcos por flota y múltiples disparos con orden. Definan claves primarias/foráneas, coordenadas dentro de rango y unicidad del disparo por atacante y destino en la partida. Normalicen jugadores y encuentros sin confundir datos completos con la vista permitida al adversario.

Maestros, operaciones e historial. Separen jugadores y torneos configurables de flotas confirmadas, disparos y resultados. ¿Qué cambia si un jugador se desactiva después de competir? ¿Puede editarse una flota ya usada? ¿Cómo se conserva un encuentro interrumpido sin alimentar ranking? Documenten restricciones sobre hechos jugados y trazabilidad de anulaciones.

Implementación e integración. Creen scripts SQL con cuatro jugadores ficticios, encuentros y secuencias para agua, impacto y hundimiento. Integren CRUD de entidades administrables y consultas históricas. Guarden flotas confirmadas y cada disparo aceptado; coordinen cierre, resultado y avance con transacciones cuando corresponda, evitando que reabrir el resultado vuelva a clasificar al ganador.

Consultas orientativas y persistencia aplicable. Historial, detalle de disparos, ranking y cuadro del torneo. Guardar flotas al confirmar su ubicación, cada disparo aceptado y el resultado al finalizar; conservar partidas interrumpidas sin sumarlas al ranking.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen ubicación/confirmación de flota y Disparar: partida activa, turno, coordenada no repetida, resultado, victoria y cambio de jugador. Incluyan ocultamiento entre turnos, intento posterior al cierre, error de persistencia y avance del ganador en el torneo.

Clases y responsabilidades. Tablero protege posiciones y Barco sus casillas; Partida controla turno y final. Un servicio de competencia coordina encuentros. Diferencien la información del dominio de los datos que se entregan a la vista rival; ocultar un color no reemplaza una selección correcta de información.

Aplicación técnica orientativa. Tablero valida coordenadas y Partida controla turnos; std::array representa casillas y std::vector la flota. Qt puede usar grillas con estados visibles. SQL relaciona jugadores, barcos, disparos y encuentros.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Barcos rectos, horizontales o verticales, completos dentro del tablero y sin superposición; se admite contacto entre barcos. No iniciar hasta completar ambas flotas. Cada turno permite un disparo válido y luego cambia de jugador, incluso si hubo impacto. Una coordenada ya atacada se rechaza sin consumir turno. Diferenciar agua, impacto y hundimiento.
- Cada jugador ve su flota y los disparos sobre ella; del rival solo ve sus disparos y resultados, sin revelar posiciones restantes. Al cambiar de turno se ocultan tableros hasta confirmar el siguiente participante. Gana quien alcanza todas las casillas de la flota rival; no se admiten disparos después del cierre. El ranking ordena por victorias, menor cantidad de derrotas y orden de registro.
- Un encuentro de torneo se registra una sola vez y su ganador ocupa la siguiente fase.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen validando una flota 6 por 6 con barcos 3, 2 y 2; luego ejecuten una secuencia corta de disparos verificable. Persistan, conecten ambas vistas y finalmente agreguen torneo/ranking. Prueben bordes, barcos en contacto permitidos, repetición de disparo y último impacto ganador.

Evidencias de funcionamiento que deben mostrar. Rechazar ubicación inválida y disparo repetido; mantener oculto el tablero rival; reconocer hundimiento y victoria; recuperar historial y avanzar de fase sin duplicar resultados.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Propongan jugadores/torneo, preparación privada, transición, tablero de partida, resultado e historial. Dibujen la navegación entre participantes y qué se oculta antes de entregar la computadora; desde el resultado debe poder consultarse el cuadro del torneo.

Qué representar en los bocetos. Muestren flota propia y disparos recibidos; en la vista rival, solo ataques realizados y resultados. Boceten barco fuera de rango, flota incompleta, coordenada repetida y partida finalizada. La transición debe ocultar ambos tableros hasta que confirme el siguiente participante.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. Los controles envían coordenadas y orientación al servicio; backend valida ubicación o disparo, persiste y devuelve solo la vista correspondiente. Qt no decide hundimiento por el color de casillas ni cambia turno después de un ataque rechazado.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-batalla-naval» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Transición privada entre turnos», terminada al ocultar tableros hasta confirmar al siguiente jugador; «Disparo repetido», terminada cuando se rechaza sin consumir turno ni guardar otro disparo.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Distingue impacto de hundimiento de barco» / «Avanza ganador de semifinal sin duplicar resultado»

Revisión integrada. Revisen preparación, vistas y registro de disparos con dos integrantes alternando turnos. Comparen secuencia visible, historial y ranking al reiniciar antes de cerrar la integración del torneo.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Presenten la finalidad recreativa y de organización de competencias locales. Expliquen la necesidad de reglas claras, ocultamiento y resultados consultables para participantes y organizadores; delimiten la flota y el torneo académicos sin justificar una actividad comercial inexistente. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Relacionen observaciones de jugadores con casos de preparación, cambio de turno, disparo y clasificación. Documenten información permitida, alternativas inválidas y resultados

esperados de agua, impacto y hundimiento. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.

- **Manual técnico.** Justifiquen representación de tablero/barcos, validación de colocación, ocultamiento, cambio de turno y cierre por flota hundida. Expliquen ranking, avance único, persistencia por disparo y tratamiento de interrupciones. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Muestren login, participantes y torneo, ubicación privada de ambas flotas, transición, disparos y cierre. Incluyan mensajes inválidos, consulta de historial/ranking y avance de fase con datos ficticios sin revelar posiciones durante la partida.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen reglas, vistas privadas y persistencia desde el primer recorrido; registren decisiones sobre datos visibles y cálculo del ranking, vinculadas con pruebas y tareas del torneo. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren barco superpuesto o fuera de borde, contacto permitido, disparo repetido, agua/impacto/hundimiento, último impacto y avance persistente sin duplicación. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.14 Ahorcado

1. Nombre del tema

Ahorcado. Juego de ahorcado con banco de palabras y pistas, niveles de dificultad, partidas individuales y competencia local por rondas con ranking.

2. Objetivo

Implementar un juego de palabras con administración del contenido y comparación de resultados. El administrador mantiene categorías, palabras y pistas; los participantes juegan y consultan estadísticas.

Alcance obligatorio y límites. Gestionar categorías, palabras y pistas; programar partidas, intentos, dificultad y puntajes; mostrar estadísticas, ranking y competencia por rondas. La modalidad académica permite partidas individuales y una competencia local de dos a cuatro jugadores con tres rondas por participante. Cada competencia utiliza una categoría y dificultad comunes.

Recorrido principal obligatorio. Cargar palabras y pistas; seleccionar categoría y dificultad; iniciar partida; ingresar letras y, si se desea, consultar la pista; finalizar por victoria o derrota; guardar puntaje. Completar una ronda de competencia y consultar acumulados y ranking.

Ampliaciones opcionales. Temporizador, más rondas, recursos sonoros o reanudación de partidas.

3. Investigación

Consulten jugadores, docentes de lengua o responsables de espacios recreativos. Observen partidas y comparen juegos de palabras para comprender dificultades de lectura, uso de pistas y presentación de resultados.

Qué observar. Sigan selección de categoría/dificultad, ingreso de letras y cierre por acierto o intentos agotados. Observen palabras con letras repetidas, acentos, Ñ, espacios o guiones y cómo se explica al participante el puntaje obtenido.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Cómo seleccionan categorías, palabras y pistas adecuadas para quienes van a jugar?
- ¿Qué debe conocer el participante antes de elegir dificultad e iniciar la partida?
- ¿Qué respuesta esperan al escribir una letra repetida o una que aparece varias veces?
- ¿Cómo entienden el tratamiento de acentos, mayúsculas, Ñ, espacios y guiones?
- ¿Cuándo les resulta útil pedir una pista y cómo debe mostrarse su efecto en el puntaje?
- ¿Qué información permite organizar rondas comparables y evitar palabras repetidas para una persona?
- ¿Qué desean consultar al terminar: errores, victorias, evolución por dificultad o posiciones de la competencia?

Materiales que conviene conocer. Revisen fichas de palabras/pistas, listados por categoría, pantallas de teclado y progreso, hojas de rondas y ranking. Registren la fuente de las observaciones y preparen un banco ficticio apropiado para la demostración.

Del hallazgo al requisito. Conviertan confusiones con letras repetidas en requisitos de entrada normalizada y devolución clara. Escriban Administrar palabra, Iniciar partida, Probar letra, Usar pista y Cerrar ronda; mantengan las dificultades y fórmula académicas aunque los juegos comparados usen otras.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Categoría contiene Palabras con pista y dificultad. Partida vincula jugador y palabra elegida, y conserva intentos, letras ingresadas y resultado. Competencia reúne participantes y rondas. La palabra utilizada y sus reglas deben conservarse o versionarse para que la edición del banco no cambie el historial.

Del análisis al modelo. Identifiquen categoría, palabra, pista, dificultad, jugador, partida, letras, competencia y ronda. Expresen categoría–palabras y competencia–participantes/rondas con sus cardinalidades. Definan claves primarias/foráneas, textos obligatorios y valores válidos de dificultad; protejan la identificación de cada ronda. Normalicen el banco y conserven contenido/reglas utilizados para explicar resultados históricos.

Maestros, operaciones e historial. Distingan banco editable de selecciones, intentos, pistas utilizadas y resultados. ¿Qué ocurre al corregir o desactivar una palabra ya jugada? ¿Cómo se preserva su forma original y su normalización? ¿Qué efectos tiene retirar un participante de una competencia iniciada? Documenten conservación o versionado y restricciones de edición.

Implementación e integración. Creen scripts SQL con palabras suficientes para tres rondas por participante y casos con acentos y Ñ. Integren CRUD del banco y consultas por dificultad. Guarden selección/reglas al iniciar, letras aceptadas y uso de pista; coordinen cierre y acumulados con transacciones cuando corresponda, sin sumar rondas interrumpidas o duplicadas.

Consultas orientativas y persistencia aplicable. Historial, porcentaje de victorias, errores por dificultad, ranking y posiciones de competencia. Guardar selección y reglas al iniciar, cada letra aceptada y uso de pista, y el resultado una sola vez al cerrar.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen Inicio con filtro y cantidad suficiente de palabras, Probar letra con normalización/repetición y Cierre con victoria o derrota. Representen uso único de pista y avance de ronda, incluyendo agotamiento de intentos, errores de escritura y desempates de competencia.

Clases y responsabilidades. Partida encapsula palabra elegida, letras utilizadas, intentos y pista; un servicio organiza rondas y evita repeticiones por participante. Separen texto mostrado y criterio de comparación para conservar Ñ y hacer consistentes todas las apariciones de una letra.

Aplicación técnica orientativa. Partida encapsula letras e intentos; un servicio organiza rondas. std::set evita repetir letras y algoritmos STL filtran palabras. Qt muestra progreso y teclado. SQL relaciona rondas, jugadores y resultados.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- La dificultad define intentos: fácil 8, media 6 y difícil 4; sus factores de puntaje serán 1, 2 y 3. Normalizar mayúsculas y vocales acentuadas, conservando la Ñ como letra distinta; espacios y guiones aparecen visibles y no se adivinan. Una letra acertada revela todas sus apariciones y no consume intento; una errónea consume uno.
- Una letra repetida se informa sin consumir ni puntuar otra vez. Gana al revelar todas las letras y pierde al agotar intentos. Puede usar una pista una sola vez. En victoria: puntaje igual a 100 por factor, menos 10 por error y 20 si usó pista, con mínimo cero; en derrota, cero.
- En competencia, cada participante recibe palabras distintas en sus rondas, con idéntica categoría y dificultad. Si faltan palabras elegibles, impedir el inicio. Desempatar por victorias, menos errores y orden de inscripción.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con una palabra fija de prueba y verifiquen revelado, error y final. Luego persistan la selección y conecten teclado; incorporen banco y competencia. Prueben último intento, letra repetida, todas las apariciones de una letra y puntaje con pista.

Evidencias de funcionamiento que deben mostrar. Resolver una palabra con letras repetidas, rechazar repetición, agotar intentos, comprobar pista y puntaje, consultar resultados tras reiniciar y completar una competencia sin duplicar rondas.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten banco de palabras/categorías, selección de modalidad y dificultad, partida, resultado y competencia/ranking. Dibujen cómo se inicia cada ronda y se consulta su detalle sin reabrirla como si estuviera activa.

Qué representar en los bocetos. Muestren letras reveladas, utilizadas, errores, intentos restantes, dificultad y disponibilidad de pista. Prevean categoría sin palabras elegibles, banco insuficiente, letra repetida, pista ya usada y partida terminada. Durante el juego no expongan la palabra completa en una tabla auxiliar.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de letra transmite la entrada al servicio; backend normaliza, valida, calcula el efecto y persiste. Qt muestra el estado devuelto y deshabilita acciones incompatibles. La pantalla de resultado consulta puntaje guardado, sin recalcularlo a partir de un banco que pudo editarse.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-ahorcado» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Normalización con Ñ distinta», terminada al tratar vocales acentuadas y preservar Ñ; «Rondas sin repetir palabra», terminada cuando cada participante recibe palabras distintas y el banco insuficiente impide iniciar.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Revela todas las apariciones de una letra acertada» / «Guarda uso único de pista y puntaje de cierre»

Revisión integrada. Acuerden normalización y contenido histórico antes de integrar el teclado. Revisen con palabras conocidas intentos, pista y puntaje; luego completen rondas y consulten acumulados tras reiniciar.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Describan la finalidad recreativa o educativa del juego y la necesidad de administrar contenido y comparar resultados. Justifiquen partidas individuales y competencia local sin atribuir beneficios de aprendizaje medidos ni plantear una venta comercial obligatoria. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Relacionen observaciones con casos del banco, partida, pista y competencia. Especifiquen actores, entradas y alternativas por repetición o banco insuficiente, y documenten la fórmula y el orden de desempate como reglas académicas. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.
- **Manual técnico.** Justifiquen normalización de acentos y Ñ, conservación/versionado de palabras, selección sin repetición, intentos y cálculo de puntaje. Expliquen cierre único, acumulados, desempates y exclusión de

partidas interrumpidas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.

- **Manual de usuario.** Muestren acceso, alta de categoría/palabra/pista, elección de dificultad, letras y pista, victoria/derrota y consulta. Continúen con inscripción, rondas y posiciones de competencia; incluyan capturas ficticias y mensajes por banco insuficiente o letra repetida.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen banco, normalización y reglas antes de la selección de rondas; registren decisiones sobre contenido histórico y suficiencia de palabras con pruebas e issues asociadas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren palabras con repeticiones, acentos y Ñ, error en el último intento, pista única, puntaje y competencia completa sin duplicar rondas; verifiquen historial tras editar una palabra del banco. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

4.15 Trivia tipo Preguntados

1. Nombre del tema

Trivia tipo Preguntados. Juego de trivia con banco de temáticas, preguntas y opciones, una modalidad local completa, resultados persistentes e historial y ranking.

2. Objetivo

Construir un juego de preguntas con banco administrable y resultados persistentes. El administrador mantiene contenido; los jugadores responden y comparan desempeño.

Alcance obligatorio y límites. Administrar temáticas, preguntas, opciones y niveles; programar partidas, respuestas y puntajes; mostrar historial y ranking. El equipo deberá explicitar su elección: modalidad individual local o multijugador local por turnos. Basta una modalidad completa. Cada partida tendrá diez preguntas; en multijugador participarán de dos a cuatro jugadores, con diez preguntas por persona y turnos alternados.

Recorrido principal obligatorio. Administrar una pregunta con opciones válidas; elegir temática y nivel; iniciar una partida con el banco disponible; responder y recibir devolución; cerrar al completar las preguntas y consultar puntaje, detalle y ranking.

Ampliaciones opcionales. Segunda modalidad, límite de tiempo, preguntas multimedia, logros o juego en red.

3. Investigación

Consulten jugadores, docentes que organicen preguntas y responsables de actividades recreativas. Comparen juegos de trivia para observar claridad de opciones, devolución, turnos y resultados, indicando qué modalidad analizaron.

Qué observar. Observen selección de temática/nivel, lectura de pregunta, confirmación de respuesta y presentación del resultado. Revisen cómo se detecta una pregunta ambigua o inválida y cómo se organizan los turnos en una computadora compartida.

Preguntas sugeridas. Utilicen estas preguntas como guía y registren las respuestas reales; pueden repreguntar para aclarar un caso concreto.

- ¿Cómo clasifican una pregunta por temática y nivel y cómo revisan que tenga una respuesta inequívoca?
- ¿Qué problemas encuentran cuando hay opciones repetidas, vacías o más de una correcta?
- ¿Qué información necesita el jugador antes de iniciar y qué debe ocurrir si faltan preguntas?
- ¿En qué momento se confirma una respuesta y qué esperan ante un segundo clic o intento de cambio?
- ¿Qué devolución ayuda a entender el acierto o error sin revelar antes la opción correcta?
- ¿Cómo comparan resultados de modalidades o dificultades diferentes y cómo representan empates?
- ¿Qué debe conservarse para revisar una partida si después se corrige una pregunta del banco?

Materiales que conviene conocer. Revisen fichas de preguntas con opciones, clasificación por tema/nivel, pantallas de respuesta, resúmenes de partidas y rankings. Pidan ejemplos autorizados y elaboren preguntas de prueba, sin presentar la comparación de juegos como entrevista inexistente.

Del hallazgo al requisito. Transformen el riesgo de cambiar una respuesta confirmada en un requisito de confirmación única. Escriban Administrar pregunta, Iniciar partida, Responder y Consultar resultado. Declaren modalidad individual o multijugador local; la investigación orienta esa elección sin exigir ambas.

El relevamiento fundamenta decisiones, pero no permite eliminar requisitos académicos ni ampliar el alcance sin control. Obtengan autorización para las evidencias y eviten datos personales o confidenciales innecesarios. Si no consiguen una visita, documenten la limitación y consulten una alternativa con la cátedra; no inventen entrevistas ni resultados.

Resultado esperado: Registro breve con fecha, contexto consultado, rol de la persona entrevistada —o fuente observada—, preguntas, hallazgos, problemas y requisitos derivados; identifiquen limitaciones y autorizaciones pertinentes.

4. Base de Datos

Partan de las operaciones relevadas y enumeren qué debe recuperarse al cerrar y reabrir. Utilicen una base relacional, preferentemente SQLite o MySQL; CSV solo complementa importación o exportación.

Entidades y relaciones orientativas. Temática agrupa Preguntas y Nivel expresa dificultad. Cada Pregunta tiene Opciones y una respuesta correcta. Partida contiene participantes y respuestas con pregunta, opción elegida, orden y puntaje. Las preguntas usadas deberán conservar su contenido o versión para no alterar resultados cuando se edite el banco.

Del análisis al modelo. Identifiquen temática, nivel, pregunta, opción, versión, partida, participante y respuesta con orden y puntaje. Expresen pregunta–opciones y partida–participantes/respuestas con cardinalidades. Usen claves primarias/foráneas, textos obligatorios y restricciones que acompañen la regla de cuatro opciones y una correcta. Normalicen categorías y niveles; documenten qué valida la base y qué completa el servicio.

Maestros, operaciones e historial. Separen banco editable de preguntas seleccionadas, respuestas confirmadas y resultados. ¿Cómo cambia una pregunta sin alterar partidas anteriores? ¿Qué ocurre al desactivar una temática utilizada? ¿Se permite corregir una respuesta ya confirmada? Conserven contenido o versión y restrinjan las modificaciones de hechos finalizados.

Implementación e integración. Creen scripts SQL y datos ficticios suficientes para la modalidad elegida, además de casos con banco insuficiente y preguntas inválidas. Integren CRUD y consultas de ranking/historial. Guarden selección/reglas al iniciar y cada respuesta confirmada; coordinen cierre y puntajes con transacciones cuando corresponda, sin contar partidas interrumpidas.

Consultas orientativas y persistencia aplicable. Ranking por nivel y modalidad, historial, aciertos por temática y preguntas con mayor proporción de errores. Guardar preguntas seleccionadas y reglas al iniciar, cada respuesta al confirmarla y puntajes al finalizar, sin sumar partidas interrumpidas.

Para cada consulta elegida documenten propósito, parámetros y resultado; compruébenla desde la aplicación. El diccionario debe explicar significado, tipo, nulabilidad, claves y restricciones de cada campo. Justifiquen las decisiones; estas entidades y consultas no constituyen un esquema definitivo ni mínimos numéricos.

Resultado esperado: DER con cardinalidades, modelo relacional, diccionario de datos, scripts SQL de creación, datos ficticios y consultas documentadas, coherentes con el CRUD, las reglas y el historial del tema.

5. Backend

Backend designa aquí la lógica y los servicios de la aplicación de escritorio en C++; no requiere un servidor web. Partan de los casos de uso y acuerden entradas, resultados, estados y responsables de cada validación.

Diagramas antes del código. Dibujen validación de pregunta, selección inicial sin repetición y Confirmar respuesta. Incluyan banco insuficiente, doble confirmación, actualización de puntaje, turno siguiente si corresponde y cierre al completar cupos. Representen errores de persistencia y empates al obtener el ranking.

Clases y responsabilidades. Pregunta protege opciones y respuesta correcta; Partida controla selección, participantes y progreso. Un servicio puede administrar el banco separado de la ejecución jugable. Justifiquen la comparación para ranking y eviten que la interfaz determine un acierto a partir del texto del botón.

Aplicación técnica orientativa. Pregunta encapsula opciones y Partida controla turnos y respuestas. std::vector reúne el conjunto y algoritmos STL permiten mezclar y ordenar. Qt presenta opciones y devolución. SQL relaciona banco, versiones, participantes y resultados.

Apliquen encapsulamiento, composición y asociaciones; utilicen herencia y polimorfismo solo si hay comportamientos que los justifiquen y expliquen la decisión. Elijan contenedores y algoritmos STL por su uso

real. Separen dominio/servicios, acceso a datos y Qt, y mantengan el diagrama de clases consistente con la implementación.

Reglas y validaciones obligatorias. Las siguientes reglas de la ficha deben reflejarse en lógica, persistencia y pruebas.

- Cada pregunta tendrá cuatro opciones y exactamente una correcta; no admitir textos vacíos ni opciones idénticas dentro de ella. Seleccionar aleatoriamente preguntas activas de la temática y nivel elegidos, sin repetición dentro de la partida. Si la modalidad multijugador necesita más preguntas que las disponibles, impedir el inicio; preparar un conjunto suficiente para todos. Todos juegan con el mismo nivel, pero con preguntas distintas.
- Una respuesta confirmada no puede cambiarse ni contabilizarse otra vez. Respuesta correcta: 10 puntos; incorrecta: 0; no se exige temporizador. La partida finaliza al completar el cupo de preguntas de cada participante. Ordenar el ranking por puntaje obtenido en partidas terminadas, separando modalidad y nivel; empates comparten posición. En multijugador, si hay empate al máximo puntaje, se declaran ganadores compartidos y se registra ese resultado.

Ante una entrada inválida, devuelvan un rechazo comprensible; ante fallas de acceso a datos, traten errores y excepciones y reviertan cambios coordinados cuando corresponda. No informen éxito ni dejen una operación parcialmente realizada.

Primer recorrido y ampliación progresiva. Comiencen con una pregunta válida de prueba y su confirmación única; después ejecuten una partida completa con selección persistida. Integren historial y ranking, y turnos si eligieron multijugador. Prueben banco justo suficiente, última respuesta, empate y edición posterior de una pregunta.

Evidencias de funcionamiento que deben mostrar. Rechazar preguntas sin respuesta correcta única; impedir iniciar con banco insuficiente; evitar repetición y doble respuesta; conservar resultados ante cambios posteriores del banco; recuperar historial al reiniciar.

Resultado esperado: Diagramas de flujo con decisiones, alternativas y errores; diagrama de clases actualizado, lógica organizada y casos de prueba válidos, inválidos y de límite que comprueben las reglas anteriores.

6. Frontend

Antes de construir Qt, dibujen en papel o digitalmente un wireframe (boceto) de cada ventana y diálogo funcional previsto. En cada boceto identifiquen propósito, campos, botones, tablas/controles, información visible, acciones, navegación, confirmaciones, validaciones, errores y estados vacíos o no disponibles.

Pantallas y navegación orientativas. Boceten banco con editor de opciones, elección de modalidad disponible/temática/nivel, partida, devolución y ranking/historial. Dibujen el retorno de resultado a consultas y, en multijugador, la transición que informa quién responde sin revelar la solución siguiente.

Qué representar en los bocetos. Muestren progreso sobre diez preguntas por participante, tema, nivel, opciones y confirmación. Prevean banco vacío o insuficiente, pregunta inválida, respuesta ya confirmada y empate final. La opción correcta se revela en la devolución, no antes; separen rankings por modalidad y nivel.

Conserven los mínimos comunes: login, QMainWindow con menú o barra de herramientas, área central, barra de estado y acceso al perfil/sesión; al menos dos ventanas o diálogos secundarios funcionales. Usen señales y slots, layouts y QSS con tema oscuro y otro de elección libre. Las pantallas sugeridas no agregan una cantidad mínima distinta de RF03.

Coordinación entre capas. El slot de Confirmar envía identificadores de pregunta y opción al servicio. Backend valida pertenencia, estado y unicidad, registra respuesta y devuelve acierto/progreso. Qt no vuelve a sumar puntos al mostrar el resultado ni permite que editar el banco cambie lo ya contestado.

Revisen navegación, mensajes, redimensionado y legibilidad en ambos temas con otro integrante. Los controles ayudan a prevenir errores; las reglas del negocio permanecen en backend y las restricciones pertinentes en la base.

Resultado esperado: Bocetos de todas las ventanas y diálogos funcionales previstos, mapa de navegación y pantallas Qt implementadas, coherentes con los casos de uso, los estados y las reglas del sistema.

7. GitLab

Creen el repositorio desde el inicio; «tp-trivia» es un nombre orientativo. Describan el objetivo y alcance elegido, registren integrantes y sus cuatro responsabilidades principales y aseguren acceso docente. GitLab y la integración son responsabilidad de todo el equipo.

Pueden conservar la organización de 3.2: src/ui/ para Qt, src/domain/ y src/services/ para dominio y casos de uso, src/data/ para persistencia, resources/ para QSS e iconos, database/ para SQL, docs/ para documentación y tests/ para pruebas o sus evidencias; main.cpp y los archivos del proyecto completan la construcción. Es orientativa, no una lista de carpetas obligatorias.

Mantengan README con objetivo, alcance, tecnologías, entorno, preparación de base y datos ficticios, compilación, ejecución y acceso exclusivamente de prueba. Configuren .gitignore para compilados, carpetas de construcción, preferencias personales y archivos temporales, conservando fuentes, recursos y scripts necesarios para reconstruir la versión.

Issues concretos. Por ejemplo: «Pregunta con correcta única», terminada al rechazar cero o varias respuestas correctas; «Selección sin repetición», terminada cuando hay preguntas distintas para todos los participantes y se bloquea un banco insuficiente.

Un issue registra una tarea: asígnenle responsable, fecha y criterio comprobable de finalización. Trabajen en ramas por funcionalidad, commits descriptivos y merge requests (solicitudes de revisión e integración). Revisen cambios, resuelvan conflictos y comprueben progresivamente un recorrido completo.

Ejemplos de mensajes de commit. «Impide modificar una respuesta ya confirmada» / «Separa ranking por modalidad y nivel»

Revisión integrada. Fijen modalidad y reglas de selección antes de conectar las pantallas. Revisen banco, contenido persistido, respuestas y ranking con una partida conocida; prueben que cambiar una pregunta después no altera el resultado histórico.

Cada integrante debe acreditar al menos tres aportes sustantivos y verificables, vinculados con tareas y resultados; contar commits no demuestra por sí solo calidad de participación. Identifiquen la entrega con tag o release, mantengan el acceso docente y respeten el cierre del 23 NOV indicado en las secciones 10 y 12.

Resultado esperado: Repositorio comprensible, ordenado y reproducible, que permita reconstruir y ejecutar el proyecto y reconocer historial, revisiones, versión de entrega y contribuciones de cada integrante.

8. Documentación

Organicen el caso de negocio y el manual técnico como partes identificables del informe de la sección 9; no necesitan crear documentos duplicados. El caso de negocio explica por qué y para quién se propone la solución; los casos de uso describen cómo interactúan actores y sistema para lograr objetivos.

- **Caso de negocio.** Presenten la finalidad recreativa o educativa y la necesidad de administrar preguntas y comparar resultados con criterios claros. Justifiquen una modalidad local completa y el alcance viable; no conviertan una comparación recreativa en una obligación comercial o servicio externo. Incluyan situación actual, problema, usuarios, necesidad, solución propuesta, beneficios esperados, alcance, límites y viabilidad académica.
- **Investigación y requisitos.** Documenten el relevamiento, la modalidad elegida y casos de edición, inicio, respuesta y resultado, con alternativas por banco insuficiente o doble confirmación. Vinculen validez de

preguntas, selección y empates con evidencias concretas. Conserven el relevamiento y describan actores, precondiciones, pasos, alternativas y resultados esperados.

- **Manual técnico.** Justifiquen conservación/versionado de contenido, selección aleatoria sin repetición, confirmación única, cupo de preguntas y cálculo de puntaje. Expliquen ranking por modalidad/nivel, posiciones compartidas, ganadores empatados cuando corresponda y partidas interrumpidas. Incluyan arquitectura, entorno, compilación/configuración, clases, diagramas de flujo, DER/modelo relacional, diccionario, scripts, reglas, errores, pruebas y limitaciones.
- **Manual de usuario.** Muestren acceso, creación de pregunta con cuatro opciones, elección de tema/nivel, partida completa, devolución y ranking. En multijugador, expliquen inscripción y turnos; incluyan capturas ficticias y mensajes por pregunta inválida o banco insuficiente.
- **Diseño de interfaz.** Incluyan los bocetos y el mapa de navegación del apartado Frontend, relacionados con los casos de uso; actualícenlos cuando cambie la solución.
- **Planificación y seguimiento.** Coordinen validez del banco, selección y persistencia antes de cerrar la interfaz; registren la elección de modalidad y el criterio del ranking con sus tareas, decisiones y pruebas. Mantengan tareas, responsables, Gantt (cronograma de tareas y duración), avances y decisiones alineados con las fechas de la sección 12.
- **Pruebas.** Registren opciones vacías/repetidas, correcta no única, banco insuficiente, selección sin repetición, doble respuesta, empate y conservación histórica después de editar el banco y reiniciar. Para cada prueba indiquen datos iniciales, pasos, resultado esperado, resultado obtenido y evidencia; registren pendientes reales.
- **Aportes individuales.** Relacionen contribuciones verificables con issues, commits o merge requests, archivos y resultados. Expliquen el trabajo de cada persona en aportes compartidos y la revisión de la versión identificada para entrega.

Resultado esperado: Documentación completa y fácil de localizar para comprender, instalar, utilizar, mantener y evaluar la versión entregada, consistente con código, datos, pantallas, pruebas y aportes individuales.

5 Roles y responsabilidad compartida

Cada integrante tendrá una responsabilidad principal entre los cuatro roles siguientes. La distribución permite coordinar el trabajo; no autoriza a desarrollar piezas aisladas que recién se reúnan al final. Todos deberán colaborar, revisar aportes de otras áreas y poder defender el sistema completo.

Rol	Responsabilidades	Evidencias	Coordinación
Backend/C++	Dominio, lógica, validaciones, excepciones, STL, servicios y pruebas.	Clases y servicios utilizados; casos válidos e inválidos; explicación de reglas y estructuras elegidas.	Acordar con Qt las entradas y salidas de cada caso de uso y con datos la consistencia de las operaciones.
Frontend/Qt	Pantallas, navegación, señales y slots, validación visual, QSS y adaptación a distintos tamaños.	Recorridos funcionales, conexiones, layouts y ambos temas legibles.	Invocar servicios existentes, mostrar sus errores y revisar con análisis que cada pantalla resuelva un caso real.
Base de datos	Modelos conceptual y lógico, tablas, claves, restricciones, SQL, CRUD, datos de prueba y respaldo.	DER, scripts reproducibles, consultas integradas, diccionario y comprobación de integridad.	Revisar reglas con backend y mantener alineados modelo, consultas, formularios y documentación.
Análisis y gestión	Investigación, alcance, requisitos, casos de uso, documentación, GitLab, Gantt, informe y manual.	Requisitos verificables, diagramas, planificación y seguimiento; documentación consistente con el producto.	Coordinar revisiones, relacionar tareas con resultados y actualizar el alcance ante decisiones del equipo.

5.1 Revisión cruzada

Como práctica orientativa, quien implementa una operación puede pedir a otro integrante que la ejecute con datos de prueba, revise el cambio y compruebe la documentación. La revisión debe producir observaciones concretas: una validación faltante, un estado incompatible, un dato que no persiste o una instrucción que no permite reproducir el resultado.

La documentación, las pruebas y el uso de GitLab son responsabilidades compartidas. El rol de análisis y gestión puede coordinarlas, pero cada integrante deberá explicar y documentar las decisiones de su trabajo. Del mismo modo, el responsable de base no es la única persona que debe comprender el modelo de datos.

En la defensa, cualquiera deberá poder seguir una operación desde la pantalla hasta la base y explicar las reglas que protege. Las contribuciones se evaluarán por su contenido, integración y dominio, no por el nombre del rol ni por la cantidad bruta de archivos modificados.

6 Requisitos funcionales obligatorios

Los siguientes requisitos se aplican a todas las opciones, incluidos los juegos. Las evidencias deben poder observarse en la aplicación y relacionarse con código y documentación.

ID	Requisito	Evidencia de cumplimiento
RF01	Acceso con inicio de sesión y validación de credenciales; como mínimo, perfil administrador. Se valorarán perfiles adicionales.	Acceso correcto con usuario de prueba, rechazo de credenciales inválidas y visualización de la sesión activa.
RF02	Ventana principal QMainWindow con menú o barra de herramientas, área central operativa, barra de estado y acceso visible al perfil o sesión.	Recorrido desde la ventana principal hacia las funciones del sistema y uso visible de sus componentes.
RF03	Al menos dos ventanas o diálogos secundarios funcionales, vinculados con casos de uso reales.	Apertura, validación y resultado útil de cada pantalla; por ejemplo, edición de una entidad y registro de una operación. Un mensaje aislado no reemplaza una pantalla funcional.
RF04	Alta, consulta, modificación y baja lógica o física de las entidades principales administrables, con confirmaciones y validaciones.	Crear, recuperar, modificar y dar de baja datos de prueba; comprobar referencias e historial cuando la baja no sea admisible.
RF05	Persistencia de datos entre ejecuciones.	Completar una operación, cerrar la aplicación, volver a abrirla y consultar el mismo resultado y sus efectos relacionados.
RF06	Mensajes claros, navegación coherente, prevención de errores, layouts y dos temas visuales: oscuro y de elección libre.	Probar datos inválidos, redimensionar y alternar temas sin perder legibilidad ni acceso a controles esenciales.
RF07	Búsquedas y reportes pertinentes, integrados en la aplicación.	Aplicar filtros, consultar resultados existentes y verificar que los totales o estados coinciden con los registros.

6.1 Criterio para bajas e historial

La **baja lógica** conserva el registro y lo marca como inactivo; la **baja física** lo elimina. La elección deberá respetar relaciones e historial. No es correcto borrar un cliente, producto o jugador si ello destruye el sentido de operaciones que deben conservarse.

Ventas, préstamos cerrados, pagos, atenciones y partidas finalizadas son registros históricos. En esos casos corresponde justificar anulación, conservación o restricción de edición. CRUD no significa permitir cambios arbitrarios sobre hechos terminados. El sistema deberá informar por qué una acción se rechaza y, cuando exista, ofrecer el procedimiento válido del dominio.

6.2 Alcance de las evidencias

Las capturas pueden complementar la entrega, pero no sustituyen una demostración funcional. Una búsqueda o reporte deberá ejecutarse desde Qt; una consulta SQL aislada en un archivo demuestra su documentación, pero no acredita por sí sola la integración requerida.

7 Evidencias específicas de Programación II

Los contenidos se evaluarán por su aplicación a un problema real del proyecto. No se exige incorporar todas las estructuras estudiadas ni agregar jerarquías o excepciones sin finalidad. Sí deberá observarse uso pertinente de POO, contenedores y algoritmos STL, manejo de errores y excepciones e integración con Qt. La STL es la biblioteca estándar de plantillas de C++, que ofrece contenedores y algoritmos reutilizables.

7.1 Diseño orientado a objetos

Obligatorio. Clases con responsabilidades claras; encapsulamiento; getters y setters justificados; composición y asociaciones correctamente modeladas; separación en archivos .h, .cpp y main.cpp. La herencia y el polimorfismo deberán utilizarse cuando el dominio los requiera, sin jerarquías artificiales. Si no resultan pertinentes, el equipo deberá explicar esa decisión.

En código. Mostrar clases que protejan su estado mediante operaciones significativas. Por ejemplo, reservar, devolver o registrarJugada permiten comprobar reglas; un setter que acepta cualquier estado puede debilitar el encapsulamiento. No es necesario proporcionar un setter para cada atributo.

En documentación. Incluir un diagrama de clases legible y explicar responsabilidades, relaciones y decisiones. Diferenciar composición, como pedido e ítems, de una asociación entre una reserva y su cliente. El diagrama debe representar clases efectivamente implementadas.

En defensa. Seguir una operación por los objetos involucrados y explicar quién mantiene cada regla. Si se utilizó polimorfismo, mostrar el comportamiento variable y por qué resulta útil. Por ejemplo, distintas políticas de cálculo pueden compartir una operación solo si realmente forman parte del alcance elegido; no deben incorporarse para cumplir una cuota de herencia.

7.2 Implementación en C++

Obligatorio. Utilizar contenedores y algoritmos STL con una finalidad concreta; validar entradas y tratar errores y excepciones; escribir funciones pequeñas y nombres significativos; evitar duplicación de lógica y variables globales; incluir comentarios que expliquen decisiones o restricciones relevantes.

En código. Identificar la colección elegida, los datos que contiene y el algoritmo que se aplica. Un vector de ítems con búsqueda, un set de letras utilizadas o un ordenamiento de posiciones son ejemplos orientativos. Copiar datos a un contenedor sin utilizarlos de forma significativa no demuestra el contenido.

En documentación. Explicar por qué la estructura elegida se ajusta al uso y describir casos de error con su respuesta esperada. Los datos de entrada inválidos deben producir mensajes comprensibles; una falla de persistencia no puede terminar en una confirmación falsa.

En defensa. Mostrar dónde se detecta, comunica y trata un error. Explicar el uso de try, throw y catch cuando corresponda y qué estado conserva el sistema al fallar. No ocultar excepciones con capturas vacías ni usarlas como reemplazo de todas las decisiones normales del programa.

7.3 Integración con Qt

Obligatorio. Señales y slots correctamente conectados; lógica del negocio separada de las pantallas; QSS con ambos temas; layouts para distintos tamaños; compilación reproducible y demostración estable.

En código. Mostrar cómo una señal, por ejemplo un clic, activa un slot que obtiene los datos, invoca un servicio y presenta su respuesta. La señal comunica un evento; el slot es la función que lo atiende. La validación de disponibilidad, total o ganador debe localizarse en la lógica, no quedar duplicada en varios botones.

En documentación. Describir navegación, pantallas y preparación del entorno. Registrar la versión de Qt, el compilador, la configuración del proyecto y el motor de datos utilizados.

En defensa. Compilar o explicar el procedimiento reproducible, realizar un recorrido estable, redimensionar una pantalla y alternar temas. Localizar la conexión entre interfaz, servicio y persistencia de la operación demostrada.

7.4 Pruebas pertinentes

Las pruebas deberán cubrir casos válidos, inválidos y límites propios del dominio. Pueden documentarse como recorridos reproducibles con datos iniciales, pasos, resultado esperado y resultado obtenido; la automatización es una mejora posible, no una tecnología obligatoria nueva. Conviene priorizar riesgos concretos: stock insuficiente, última plaza, doble confirmación, fecha superpuesta, letra repetida o resultado de una partida ya cerrada.

8 Evidencias específicas de Base de Datos I

El modelo de datos deberá permitir que la aplicación conserve información coherente y la recupere con sentido. No se impone una cantidad arbitraria de tablas o consultas: la suficiencia se evaluará respecto del alcance y las relaciones del tema asignado.

8.1 Modelo conceptual

Obligatorio. Representar entidades, atributos, relaciones, cardinalidades y reglas del negocio en un DER legible. El diagrama entidad-relación describe qué información existe y cómo se vincula. La cardinalidad indica cuántas ocurrencias pueden participar en una relación.

Evidencia. Explicar, por ejemplo, por qué una obra tiene varios ejemplares o por qué una función tiene muchas butacas disponibles pero una asignación vigente no puede repetirse. Identificar relaciones opcionales y obligatorias. Los ejemplos orientan; el equipo debe completar su propio modelo.

8.2 Modelo lógico e integridad

Obligatorio. Definir tablas, claves primarias y foráneas, restricciones y normalización suficiente; justificar las decisiones. La clave primaria identifica un registro, la foránea enlaza registros y una restricción protege condiciones válidas. La normalización busca evitar repeticiones que generen contradicciones al modificar datos.

Evidencia. Mostrar cómo se traducen las relaciones, incluidas las de muchos a muchos cuando existan, y cómo se controlan campos obligatorios, unicidad y valores permitidos. Explicar qué ocurre al modificar o dar de baja información relacionada. Los precios aplicados y otros valores históricos pueden conservarse como parte de una operación; deberán distinguirse de datos maestros actualizables.

8.3 Implementación SQL

Obligatorio. Entregar scripts de creación, restricciones, datos de prueba y consultas relevantes. El diccionario describirá para cada campo su significado, tipo, admisión de nulos, clave o restricción y relaciones pertinentes. SQL es el lenguaje utilizado para definir, consultar y modificar la información relacional.

Evidencia. Recrear la base siguiendo el README y ejecutar operaciones sin depender de una copia instalada solo en la computadora de un integrante. Documentar propósito, parámetros y resultado de las consultas principales. Combinaciones mediante JOIN, filtros y agrupaciones son recursos orientativos; deberán responder a preguntas reales del proyecto.

8.4 Integración y cambios consistentes

Obligatorio. Implementar CRUD, búsquedas y reportes desde C++/Qt, controlar errores y mantener integridad ante operaciones inválidas. Cuando una operación modifica varias tablas que deben mantenerse coordinadas, tratar esos cambios de manera consistente, usando transacciones cuando corresponda.

Orientaciones. Una venta necesita cabecera, detalles y stock coherentes; un pago dividido debe respetar el saldo; finalizar una partida puede cerrar su estado y habilitar el siguiente encuentro. Si una escritura falla, no deben quedar fragmentos confirmados como si la operación hubiese terminado. Una consulta previa de disponibilidad, por sí sola, no protege frente a una segunda asignación: combinar validación y mecanismos de integridad adecuados al motor.

Se recomienda parametrizar las consultas para separar datos ingresados de las instrucciones SQL y centralizar la conexión y sus errores. SQLite y MySQL requieren configurar correctamente sus mecanismos de integridad; el equipo deberá demostrar que las restricciones utilizadas están efectivamente activas.

8.5 Comprobaciones de integridad y persistencia

Preparar datos ficticios que permitan mostrar un caso válido y los conflictos relevantes del tema. Entre las comprobaciones orientativas se encuentran intentar una referencia inexistente, duplicar una asignación única, borrar un registro referenciado o provocar el rechazo de una operación que modificaría varias tablas.

Para cada caso, observar el mensaje y consultar el estado posterior: no basta con que la aplicación no se cierre. Después de una operación exitosa, reiniciar y verificar sus relaciones y totales. Después de un rechazo, comprobar que no se guardaron cambios parciales. Documentar también cómo respaldar y recuperar los datos de prueba utilizados en la demostración.

9 Entregables

La entrega deberá permitir comprender, preparar, ejecutar y revisar el sistema desde el repositorio. Las ubicaciones siguientes son sugeridas; pueden adaptarse si el README permite localizar claramente cada componente.

Entregable	Contenido obligatorio	Ubicación sugerida
Repositorio GitLab y README	Título, integrantes, comisión, descripción, tecnologías, instrucciones de compilación y ejecución y credenciales exclusivamente de prueba. Mantener un README inicial y actualizarlo para la entrega.	Raíz del repositorio; README.md.
Código fuente	Proyecto completo y ordenado, archivos necesarios para compilar y recursos, sin binarios innecesarios. Versión entregable identificada con tag o release.	src/, resources/ y archivos del proyecto en la raíz.
Informe técnico	Caso de negocio, investigación, requisitos y casos de uso; manual técnico con arquitectura, entorno, decisiones, diagramas de clases y flujo, datos, reglas, errores, pruebas, límites y conclusiones. Contenido detallado en el apartado Documentación de cada ficha, como partes del mismo informe.	docs/informe/
Base de datos	DER, modelo relacional, diccionario, creación SQL, inserciones de prueba y consultas principales documentadas.	database/ y docs/datos/
Manual de usuario	Instalación, inicio de sesión, navegación, operaciones principales, mensajes frecuentes y capturas coherentes con la versión final.	docs/manual/
Planificación	Gantt, distribución de tareas, hitos, riesgos, seguimiento y registro de cambios relevantes. El Gantt muestra tareas, duración y relación temporal.	docs/planificacion/ e issues.
Evidencias individuales	Contribuciones identificables relacionadas con tareas, decisiones, resultados y revisión.	Historial, issues, merge requests y docs/aportes/.
Diseño de interfaz	Bocetos de cada ventana y diálogo funcional previsto, con acciones, estados y mensajes; mapa de navegación coherente con casos de uso y pantallas finales.	docs/interfaz/ o sección identificable del informe.

9.1 Reproducibilidad desde el repositorio

El README deberá explicar cómo obtener la versión entregada, qué entorno utilizar, cómo configurar la conexión, ejecutar los scripts, cargar datos ficticios, compilar e iniciar sesión. Si hay pasos manuales, deberán estar descritos de manera que otra persona pueda seguirlos.

Las credenciales publicadas serán **exclusivamente de prueba**. La configuración no deberá depender de rutas personales sin explicar ni de archivos esenciales ausentes. El repositorio debe contener los scripts necesarios para recrear la base; una captura o un archivo de base ya cargado pueden complementar, pero no sustituir ese procedimiento.

La documentación deberá coincidir con la versión identificada para entrega. Los diagramas no deben describir módulos inexistentes, ni el manual mostrar funciones que no están disponibles. Un registro de pruebas útil especifica qué se hizo, con qué datos y qué resultado se obtuvo, incluyendo los errores encontrados y su resolución o limitación conocida.

10 Criterios mínimos para GitLab

GitLab deberá utilizarse desde el inicio. El historial permitirá reconocer cambios, responsables, motivos y resultados integrados al producto.

10.1 Flujo de trabajo obligatorio

1. **Planificar con issues o tareas**, con responsable y fecha, vinculadas con requisitos, errores o mejoras del alcance.
2. **Desarrollar en ramas por funcionalidad o ajuste.** Cada rama aísla cambios hasta su integración a la versión compartida.
3. **Registrar commits pequeños, descriptivos y atribuibles**, con mensajes que permitan reconocer la finalidad del cambio.
4. **Integrar mediante merge requests y resolver conflictos.** Estas solicitudes permiten revisar e incorporar cambios; conservar las decisiones relevantes.
5. **Entregar README final, tag o release y repositorio completo antes de la presentación**, con cierre el 23 NOV.

10.2 Participación mínima y aportes sustantivos

Cada integrante deberá contar con **al menos tres aportes sustantivos y verificables**. Cada aporte produce un resultado útil, localizable en el historial y explicable en su necesidad, implementación y validación; puede abarcar varios commits o archivos.

Rol principal	Ejemplo orientativo de aporte sustantivo	Cómo comprobarlo
Backend/C++	Implementar reservas con validación de superposición y pruebas.	Issue, código, revisión y ejecución de casos válidos e inválidos.
Frontend/Qt	Crear alta y edición integradas con servicios, layouts, mensajes y temas.	Merge request, pantallas funcionales, conexiones y prueba del recorrido.
Base de datos	Diseñar relaciones y restricciones e integrar una consulta útil.	DER, diccionario, scripts e historial; prueba de integridad y consulta.
Análisis y gestión	Elaborar casos de uso verificables y ajustar manual y pruebas tras revisar el sistema.	Versiones, decisiones y tareas vinculadas con documentación utilizada.

Los ejemplos no restringen aportes a un rol. No alcanzan cambios cosméticos, archivos vacíos ni cargas masivas al final. **Tres commits no equivalen automáticamente a tres aportes**: fragmentar un trabajo no crea resultados sustantivos independientes. El conteo no reemplaza contenido ni dominio individual. En resultados compartidos, explicar la contribución de cada persona. Análisis y documentación también pueden constituir aportes sustantivos verificables.

10.3 Cierre e identificación de la entrega

El **cierre del repositorio** identifica y congela la versión entregada mediante tag o release, conservando acceso docente e historial. Un tag señala un estado concreto; una release puede asociarle descripción y materiales. No se deberá mover esa referencia para ocultar cambios posteriores: estos quedarán diferenciados de la entrega. La política de aceptación de cambios tardíos corresponde a la cátedra y no se establece aquí.

11 Rúbrica porcentual propuesta

La rúbrica propone seis criterios cuya suma es **100%**. Los niveles describen calidad observable; no fijan una nota mínima, descuentos automáticos ni una conversión numérica adicional. Las ampliaciones opcionales no sustituyen el alcance obligatorio.

Criterio	Ponderación	Qué se observa
Funcionamiento e integración	25%	Casos de uso completos, estabilidad, validaciones y persistencia.
Programación II	20%	POO, herencia justificada cuando corresponda, STL, estructura, legibilidad y Qt.
Base de Datos I	20%	Modelo, integridad, SQL, CRUD, consultas e integración.
Análisis y documentación	15%	Requisitos, casos de uso, diagramas, informe, manual y Gantt.
GitLab y trabajo en equipo	10%	Trazabilidad, aportes individuales, organización y cumplimiento.
Presentación oral	10%	Claridad, síntesis, demostración y respuestas del equipo.
Total	100%	Producto y proceso, con evidencias grupales e individuales.

11.1 Funcionamiento e integración 25%

Nivel	Descriptor observable
Excelente	Completa el alcance obligatorio con recorridos coherentes entre interfaz, lógica y base; controla casos inválidos y límites relevantes; conserva datos e historial y demuestra estabilidad sin intervención manual para corregir resultados.
Satisfactorio	Los casos centrales y requisitos comunes funcionan integrados; las validaciones y la persistencia se comprueban. Presenta detalles menores que no impiden los recorridos ni comprometen la coherencia de los datos.
En desarrollo	Algunos recorridos funcionan, pero faltan funciones del alcance o existen pasos manuales, errores frecuentes o validaciones incompletas. La persistencia se verifica solo de manera parcial.
Insuficiente	No logra demostrar el proceso principal completo, pierde datos o produce estados incoherentes; la aplicación presenta fallas que impiden utilizarla para el propósito declarado.

11.2 Programación II 20%

Nivel	Descriptor observable
Excelente	Clases y relaciones expresan el dominio con encapsulamiento efectivo; justifica herencia y polimorfismo cuando corresponden. Usa STL con sentido, trata errores y excepciones, mantiene código legible y separación de capas, y demuestra Qt con señales, slots, layouts y ambos temas.
Satisfactorio	Aplica POO y STL de manera pertinente, separa responsabilidades y conecta Qt con los servicios. El tratamiento de errores y la estructura son comprensibles; las decisiones sobre herencia están justificadas y los requisitos de interfaz se cumplen.
En desarrollo	El diseño mezcla responsabilidades o duplica lógica; el uso de STL, las excepciones o la integración Qt son limitados. Hay encapsulamiento débil, relaciones poco claras o requisitos visuales incompletos.
Insuficiente	El código no evidencia los contenidos centrales o concentra el negocio en las pantallas; no puede justificar su diseño ni demostrar uso pertinente de STL, manejo de errores e integración Qt.

11.3 Base de Datos I 20%

Nivel	Descriptor observable
Excelente	DER, modelo lógico, scripts y aplicación coinciden; claves y restricciones protegen reglas, las operaciones relacionadas son consistentes y las consultas responden al dominio. La base se recrea y la integridad y persistencia se demuestran con casos válidos e inválidos.
Satisfactorio	Modelo suficiente y justificado, CRUD y consultas integrados, scripts reproducibles y controles de integridad efectivos para los procesos centrales. La documentación permite comprender tablas y relaciones.
En desarrollo	Existen discrepancias entre modelo y tablas, restricciones incompletas, consultas poco integradas o pasos no documentados para recrear la base. Algunos errores dejan relaciones o cambios inconsistentes.
Insuficiente	No existe persistencia relacional funcional o el modelo no sostiene el proceso; CRUD y consultas no se integran y no se puede recrear la base ni demostrar integridad.

11.4 Análisis y documentación 15%

Nivel	Descriptor observable
Excelente	Problema y alcance precisos; requisitos verificables y casos de uso completos. Diagramas, informe, pruebas, manual y Gantt son claros, se relacionan entre sí y reflejan la versión entregada y sus decisiones.
Satisfactorio	Los documentos requeridos están disponibles y permiten comprender, preparar y utilizar la aplicación. Hay coherencia general entre alcance, diseño, pruebas y planificación, con omisiones menores.
En desarrollo	La documentación es parcial, desactualizada o poco verificable; faltan relaciones entre requisitos, pruebas y decisiones. El manual o la planificación requieren aclaraciones para poder utilizarse.
Insuficiente	Faltan documentos esenciales o describen un producto diferente; no permiten conocer alcance, preparar la aplicación, utilizarla ni explicar las decisiones del equipo.

11.5 GitLab y trabajo en equipo 10%

Nivel	Descriptor observable
Excelente	Historial progresivo con tareas, ramas, commits y merge requests trazables; revisiones y conflictos resueltos con criterio. Cada integrante acredita al menos tres aportes sustantivos y comprende su integración; el cierre conserva una versión inequívoca y documentación completa.
Satisfactorio	Utiliza el flujo requerido y permite identificar tareas y resultados. Cada integrante acredita al menos tres aportes sustantivos; el repositorio, README y tag o release de entrega son localizables, con coordinación y cumplimiento verificables.
En desarrollo	Historial o planificación incompletos, integración poco trazable o contribuciones difíciles de atribuir. Falta evidencia suficiente del mínimo de aportes de algún integrante o de la identificación clara de la entrega.
Insuficiente	Predomina una carga final sin proceso verificable, no se pueden identificar contribuciones sustantivas ni coordinación y no hay una versión entregada claramente identificada.

11.6 Presentación oral 10%

Nivel	Descriptor observable
Excelente	Exposición clara y ajustada a diez minutos, con participación de todos. La demostración reproduce el proceso, la persistencia y un caso inválido; el equipo justifica decisiones, reconoce límites y responde sobre el sistema completo.
Satisfactorio	Presenta problema, diseño, demostración y decisiones en el tiempo previsto; participan todos y las respuestas muestran comprensión del funcionamiento y de las responsabilidades principales.
En desarrollo	La exposición pierde foco o distribución del tiempo; la demostración es parcial y las respuestas dependen de un único integrante o no explican algunas decisiones del sistema.
Insuficiente	No logra comunicar el propósito ni demostrar el funcionamiento; la participación y las respuestas no permiten acreditar comprensión del trabajo presentado.

11.7 Producto grupal y evidencia individual

El producto grupal se analizará con los seis criterios y ponderaciones indicados. La evidencia individual comprenderá aportes identificables, participación en revisiones, comprensión de las decisiones y respuestas durante la defensa. Ningún rol exime de entender el sistema completo.

La conversión de estas evidencias en una calificación individual, si requiere un mecanismo adicional, será una **Propuesta a confirmar por la cátedra**. Esta consigna no establece una fórmula definitiva, un porcentaje adicional ni descuentos automáticos. Se mantienen las ponderaciones propuestas y no se fija una nota mínima de aprobación.

12 Cronograma de avances

Las fechas se expresan con día y mes, sin asignación de día de la semana. Los hitos indican el foco de revisión; modelado, interfaz, base e integración deberán trabajarse coordinadamente desde el inicio.

Fecha	Hito	Evidencias esperadas	Criterio de revisión
30 OCT	Investigación + GitLab	Tema, problema, alcance, requisitos iniciales, casos de uso preliminares, repositorio y planificación. Proyecto base que compile y ejecute.	Alcance viable para cuatro integrantes; organización visible y punto de partida reproducible, sin exigir funciones finales.
06 NOV	Backend	Modelo de clases, diagramas de flujo de procesos centrales, lógica principal, estructuras STL, validaciones y avance funcional sin depender de la interfaz final.	Reglas comprobables mediante un recorrido ejecutable, diseño comprensible y documentación alineada.
13 NOV	Base de datos	DER, modelo relacional, scripts, datos de prueba, CRUD y primeras consultas integradas.	Base recreable; relaciones e integridad coherentes; operaciones desde la aplicación con persistencia verificable.
20 NOV	Frontend	Bocetos, mapa de navegación y pantallas Qt, navegación, señales y slots, validación visual, QSS y dos temas.	Recorridos integrados, uso claro, separación del negocio y controles legibles al redimensionar y cambiar de tema.
23 NOV	Cierre del repositorio	README, informe con caso de negocio y manual técnico, manual de usuario, evidencias individuales, versión final y preparación de la demostración.	Tag o release inequívoco; archivos completos, instrucciones reproducibles y coincidencia entre documentación y producto.
27–28 NOV	Presentación final	Exposición oral, demostración funcional y defensa técnica.	Síntesis en diez minutos, participación de todos, funcionamiento demostrado y comprensión de decisiones.
Primeros días de diciembre	Recuperatorio, a coordinar	Evidencias correspondientes a los aspectos que indique la cátedra.	Fecha y organización a coordinar; no se fija aquí un día adicional.

12.1 Qué deberá mostrar cada avance

Cada hito requiere **evidencia ejecutable y documentación actualizada**, acordes con su etapa. El 30 OCT basta un proyecto base que compile y se inicie, sin funcionalidades finales anticipadas.

Para backend puede utilizarse una pantalla provisoria o un ejecutor de casos que invoque servicios. Los avances posteriores mostrarán progresivamente persistencia, consultas e interfaz integradas. Capturas y diagramas complementan la ejecución.

GitLab y el Gantt deberán reflejar avances, pendientes, riesgos y cambios. Los hitos son focos de revisión: base, Qt y lógica deben coordinarse desde el inicio.

13 Organización de la exposición de diez minutos

La exposición deberá respetar la siguiente distribución. El recorrido se preparará con datos ficticios y pasos reproducibles para mostrar el sistema dentro del tiempo disponible.

Tiempo	Bloque	Contenido
1 minuto	Problema y alcance	Necesidad, destinatarios y límites del proyecto.
2 minutos	Diseño	Casos de uso, arquitectura, clases principales y modelo de datos.
4 minutos	Demostración	Login, operación central, persistencia, validación y reporte o consulta.
2 minutos	Decisiones técnicas	POO, STL, Qt, SQL, integridad y desafíos resueltos.
1 minuto	Cierre	Resultados, limitaciones, mejoras futuras y aprendizaje.
10 minutos	Total	Participación de los cuatro integrantes.

13.1 Reparto orientativo entre integrantes

Bloque	Reparto sugerido	Duración
Problema y alcance	Integrante 1 presenta la necesidad y delimita el proyecto.	1 minuto
Diseño	Integrante 1 explica casos de uso y arquitectura; integrante 2 muestra clases y modelo de datos.	1 minuto cada uno
Demostración	Integrante 3 realiza login y comienza la operación central; integrante 4 la completa y muestra persistencia, validación y consulta.	2 minutos cada uno
Decisiones técnicas	Integrante 2 explica POO, STL, SQL e integridad; integrante 3 explica Qt y un desafío de integración.	1 minuto cada uno
Cierre	Integrante 4 presenta resultados, límites y aprendizaje.	1 minuto

Este reparto suma diez minutos: integrantes 1 y 2 participan dos minutos cada uno; integrantes 3 y 4, tres cada uno. Puede adaptarse sin modificar los tiempos de los bloques ni excluir participantes. Los roles no limitan qué puede exponer cada persona.

13.2 Preparación de la demostración

Ensayar con la versión entregada y datos ficticios preparados. Realizar login y una operación breve; cerrar y reabrir para mostrar persistencia; provocar una validación y ejecutar una consulta.

Los datos precargados pueden preparar catálogos, operaciones anteriores o fases de competencia. En todos los casos se deberá ejecutar el proceso central; en los juegos, una secuencia jugable con efecto persistente.

Todos deberán estar preparados para responder sobre el sistema completo, localizar decisiones en el código y explicar sus limitaciones. No se establece aquí una duración adicional para preguntas.

14 Lista final de verificación

Antes de identificar la versión de entrega, revisen cada punto con el sistema y el repositorio disponibles. Las casillas permiten registrar la comprobación del equipo.

☐ El sistema compila y ejecuta, indicado por la cátedra, y el README permite reproducirlo.

☐ Funcionan el login, la ventana principal QMainWindow y al menos dos ventanas o diálogos secundarios vinculados con casos de uso reales.

☐ Los casos de uso principales de la opción asignada funcionan completos con datos persistentes; se comprobó el cierre y reapertura de la aplicación.

☐ Las validaciones impiden operaciones inválidas y muestran mensajes claros, sin confirmar cambios parciales como exitosos.

☐ La base conserva integridad y puede recrearse mediante los scripts y datos de prueba del repositorio.

☐ Se evidencian POO, STL, Qt y los contenidos de Base de Datos I; las decisiones de herencia y polimorfismo están justificadas cuando corresponden.

☐ Funcionan el tema oscuro y el tema de elección libre; las pantallas mantienen legibilidad y adaptación a distintos tamaños.

☐ GitLab permite identificar al menos tres aportes sustantivos y verificables por integrante, relacionados con tareas y resultados.

☐ El repositorio contiene el README final, los componentes de la entrega y el tag o release que identifica la versión congelada.

☐Informe con caso de negocio y manual técnico, investigación y requisitos, modelo de datos, diccionario, pruebas, Gantt y manual de usuario coinciden con la versión entregada y pueden localizarse desde el README.

☐Se elaboraron diagramas de flujo de los procesos centrales con alternativas y errores; el diagrama de clases coincide con la implementación.

☐Están los bocetos de cada ventana y diálogo funcional previsto y el mapa de navegación, coherentes con los casos de uso y las pantallas entregadas.

☐ La demostración está ensayada con datos ficticios, participan todos y cabe en diez minutos.

Revisión del alcance. Las funcionalidades de la ficha asignada y los requisitos comunes deben estar presentes. Las ampliaciones opcionales están diferenciadas y no sustituyen obligaciones. Los juegos incluyen lógica jugable, administración, persistencia e historial; los proyectos de gestión conservan relaciones y operaciones históricas coherentes.

Revisión compartida. Cada integrante deberá poder explicar un recorrido completo entre pantalla, lógica y base y reconocer las limitaciones conocidas. La entrega debe representar el trabajo coordinado del equipo y el aprendizaje individual de sus integrantes.

El objetivo final es integrar conocimientos para entregar una solución útil, defendible y profesional.